Sam's Teach Yourself C++17 Notes

Contents

1 Getting Started **5**
- 1.1 History of C++ . . . 5
- 1.2 Programming a C++ Application . . . 6
  - 1.2.1 Steps to Generating an Executable . . . 6
  - 1.2.2 Programming Your First C++ Application . . . 7
- 1.3 QA . . . 9
- 1.4 Quiz . . . 10
- 1.5 Exercies . . . 11

2 The Anatomy of a C++ Program **12**
- 2.1 Parts of the Hello World Program . . . 12
- 2.2 Preprocessor Directive #include . . . 13
- 2.3 The Body of Your Program main() . . . 14
- 2.4 Returning a Value . . . 15
- 2.5 Namespaces . . . 16
- 2.6 Functions in C++ . . . 18
- 2.7 Basic Input Using std::cin and Output using std::cout . . . 20
- 2.8 QA . . . 21
- 2.9 Quiz . . . 22
- 2.10 Exercises . . . 23

3 Using Variables, Declaring Constants **24**
- 3.1 What Is a Variable? . . . 24
  - 3.1.1 Memory and Addressing . . . 24
  - 3.1.2 Declaring Variables to Access and Use Memory . . . 25
  - 3.1.3 Declaring and Initializing Multiple Variables of a Type . . . 26
  - 3.1.4 Scope of a Variable . . . 27
  - 3.1.5 Global Variables . . . 28
- 3.2 Common Compiler-Supported C++ Variable Types . . . 29
  - 3.2.1 Signed vs Unsigned integers . . . 30
  - 3.2.2 Signed integer types: short, int, long, and long long . . . 32
  - 3.2.3 Unsigned integer types: unsigned short, unsigned int, unsigned long, unsigned long long . . . 33
  - 3.2.4 Integer Overflow . . . 34
  - 3.2.5 Floating-Point types `float` and `double` . . . 35
- 3.3 Size of a Variable using sizeof . . . 37
- 3.4 Automatic Type Inference Using auto . . . 38
- 3.5 typdef and/or using to Substitute a Variable's Type . . . 39
- 3.6 What is a Constant? . . . 40
  - 3.6.1 Literal constants . . . 40
  - 3.6.2 Declaring Variables as Constants Using const . . . 42
  - 3.6.3 Constant Expression Using constexpr . . . 43
  - 3.6.4 Enumerations . . . 45
  - 3.6.5 Defining Constants Using #define . . . 47
- 3.7 Keywords that Cannot be Used as Variable or Constant Names . . . 48
- 3.8 QA . . . 49
- 3.9 Quiz . . . 50

4 Managing Arrays and Strings **52**
4.1 What is an Array? . . . 52
4.1.1 The Need for Arrays . . . 53
4.1.2 How Data Is Stored in an Array . . . 54
4.1.3 Accessing Data Stored in an Array . . . 55
4.1.4 Modifying Data Stored in an Array . . . 56
4.2 Multidimensional Arrays . . . 57
4.3 Dynamic Arrays . . . 58
4.4 C-style Character Strings . . . 60
4.4.1 Difference between `'\n'` and `std::endl` . . . 61
4.5 C++ Strings: std::string . . . 62
4.6 QA . . . 63
4.7 Quiz . . . 64

5 Working with Expressions, Statements, and Operators **66**
5.1 Statements . . . 66
5.2 Compound Statements or Blocks . . . 67
5.3 Using Operators . . . 68
5.3.1 Assignment Operator (=) . . . 68
5.3.2 L-vales and R-values . . . 68
5.3.3 Operators to Add (+), Subtract(-), Multiply(* or <<), Divide(/ or >>), and Modulo(%) . . . 69
5.3.4 Operators to Increment(++) and Decrement(–) . . . 70
5.3.5 Equality Operators (==) and (!=) . . . 71
5.3.6 Relational Operators . . . 73
5.3.7 Logical Operations NOT, AND, OR, and XOR . . . 74
5.3.8 Bitwise Operators: NOT (~), AND (&), OR (|), XOR (^) . . . 75
5.3.9 Bitwise right shift >> and left shift << . . . 76
5.3.10 Compound Assignment Operators . . . 78
5.3.11 Using sizeof Operator . . . 79
5.3.12 Operator Precedence . . . 80
5.4 QA . . . 81
5.5 Quiz . . . 82

6 Controlling Program Flow **84**

7 Organizing Code with Functions **84**

8 Pointers and References **86**
8.1 What is a Pointer? . . . 86
8.1.1 Declaring a Pointer . . . 87
8.1.2 Determing the Address of a Variable Using the Reference Operator (&) . . . 88
8.1.3 Access Pointer Data Using the Dereference Operator (*) . . . 89
8.1.4 What is sizeof() a Pointer? . . . 90
8.2 Dynamic Memory Allocation . . . 91
8.2.1 Using new and delete or delete[] . . . 91
8.2.2 Using malloc, realloc, etc and free . . . 92
8.2.3 Operators vs Functions vs constexpr . . . 96
8.2.4 Operator Overloading Examples (Unary, Binary, and Ternary-like) . . . 99
8.2.5 Incrementing (++) and Decrementing (−− on Pointers) . . . 102
8.2.6 Using the const keyword on Pointers . . . 104
8.2.7 Passing by ref C vs C++ . . . 106
8.2.8 Clarification for pointer declarations . . . 107
8.2.9 Passing Pointers to Functions . . . 110
8.2.10 Pointers in Static vs Dynamic Memory . . . 111
8.3 Common Programming Mistakes when using Pointers . . . 113
8.4 References . . . 114
8.4.1 References as Function Parameters . . . 115
8.4.2 References vs Pointers as Function Arguments . . . 117

8.4.3 Using const on References . . . 119
8.4.4 Extended: Using const with References . . . 120
8.4.5 Definition of Reference in C++ and why it's not valid in C . . . 121
8.4.6 QA . . . 122
8.4.7 Quiz . . . 124

9 Classes and Objects **126**
9.1 Classes . . . 126
9.1.1 An Object as an Instance of a Class . . . 126
9.1.2 Accessing Members Using the Dot Operator (.) . . . 127
9.1.3 Accessing Members Using the Pointer Operator (->) . . . 128
9.1.4 Class vs Struct . . . 130
9.1.5 Different Ways to Declare a `struct` . . . 131
9.1.6 Public, Private, Protected, Friend in C++ . . . 133
9.1.7 This Keyword . . . 135
9.1.8 Static Variables in Classes . . . 136
9.2 Constructors . . . 137
9.2.1 Declaring and Implementing a Constructor . . . 137
9.2.2 When and How to Use Constructors . . . 138
9.2.3 Overloading Constructors . . . 138
9.2.4 Class Without a Default Constructor . . . 138
9.2.5 Constructor Parameters with Default Values . . . 139
9.2.6 Constructors with Initialization Lists . . . 140
9.3 Destructor . . . 142
9.3.1 Declaring and Implementing a Destructor . . . 142
9.3.2 When and How to Use a Destructor . . . 143
9.4 Copy Constructors . . . 144
9.4.1 Shallow Copying and Associated Problems . . . 144
9.4.2 Ensuring Deep Copy Using a Copy Constructor . . . 146
9.4.3 Explicit keyword . . . 150
9.4.4 Move Constructors Help Improve Performance . . . 151
9.5 Different Uses of Constructors and Destructors . . . 154
9.5.1 Class That Does Not Permit Copying . . . 154
9.5.2 Singleton Class That Permits a Single Instance . . . 155
9.6 Class That Prohibts Instantiation on the Stack . . . 156
9.6.1 Using Constructors to Convert Types . . . 157
9.7 sizeof() a Class . . . 160
9.8 How struct differs from class . . . 164
9.9 Declaring a 'friend' of a class . . . 166
9.10 union: A Special Data Storage Mechanism . . . 167
9.11 Using Aggregate Initialization on Classes and Structs . . . 168
9.12 constexpr with Classes and Objects . . . 170
9.13 QA . . . 171
9.14 Quiz . . . 172

10 Implementing Inheritance **174**
10.1 Basics of Inheritance . . . 174
10.1.1 Access specific keyword protected . . . 176
10.1.2 Base Class Initialiation — Passing Parameters to the Base Class . . . 177
10.1.3 Derived Class Overriding Base Class's Methods . . . 178
10.1.4 Invoking Overriden Methods of a Base Class . . . 179
10.1.5 Derived Class Hiding Base Class's Methods . . . 180
10.2 Order of Construction . . . 181
10.3 Order of Destruction . . . 182
10.4 Private Inheritance . . . 184
10.5 Composition (side note) . . . 186
10.6 Protected Inheritance . . . 187
10.7 Multiple Inheritance . . . 189

10.8 Avoid Inheritance via `final` . . . 191

11 Polymorphism **194**
11.1 Basics of Polymorphism . . . 194
11.1.1 Need for Polymorphic Behavior . . . 194
11.1.2 Polymorphic Behavior Implemented Using Virtual Functions . . . 195
11.1.3 Need For Virtual Destructors . . . 196
11.1.4 How do Virtual Functions Work? — The Virtual Function Table (vtable) . . . 198
11.2 Abstract Base Classes and Pure Virtual Functions . . . 201
11.3 Using 'virtual' Inheritance to Solve the Diamond Problem . . . 202
11.4 Specifier 'override' to Indicate Intention to Override . . . 205
11.5 Using 'final' to Prevent Function Overriding . . . 207
11.6 Virtual Copy Constructors? . . . 208
11.7 QA . . . 209
11.8 Quiz . . . 210

12 Operator Types and Operator Overloading **212**
12.1 What are Operators in C++? . . . 212
12.2 Unary Operators . . . 214

1 Getting Started

1.1 History of C++

- Initially developed by Bjarne Stroustroup at Bell Labs in 1979.
- It was meant to be a successor to C.
- C++ was designed to be an **object-oriented** language with concepts like
  - inheritance
  - abstraction
  - polymorphism
  - encapsulation
- In 1998, the first standward of C++ was ratified by the ISO Committee.
  (This is when we get C++14 for example for the year it is ratified (2014) again and so on)
- C++ can be for operating systems, web services, databases, and much more.

1.2 Programming a C++ Application

When you start Notepad on Windows or a terminal on Linux, you are running an ***executable*** of that program. The **executale** is the finished product that can be run.

1.2.1 Steps to Generating an Executable

1. **Writing** (or programming) C++ code using a text editor/IDE.
2. **Compiling** code using a C++ ***compiler*** that converts it to a **machine language** version contained in "***object files***"
   (gcc (gnu compiler collection) or clang (llvm) or msvc (microsoft visual c++), etc)

   gcc example: g++ main.cpp -o main
   clang example: clang++ main.cpp -o main
   msvc example: cl main.cpp
3. **Linking** the output of the compiler using a ***linker*** to get an ***executable*** (.exe for Windows, for example)

```
main.cpp
   |
   v
Compiler (g++, clang++, MSVC)
   |
   v
main.o (Linux/macOS) or main.obj (Windows)
   |
   v
Linker
   |
   v
program.exe (Windows)
program     (Linux/macOS)
```

Compilation is the step where code in C++ (typically text files with .cpp extension) is converted into ***byte code*** that the processor can **execute**.

The ***Compiler*** converts one code file at a time, generating an **object file** with a **.o** (linux) or **.obj** (Windows) extension, **ignoring dependencies** that this cpp file may have on code in another file.

The ***Linker*** joins the dots and resolves these **dependencies**.

In the event of successful **linkage**, it creates an **executable** for the programmer to execute and distribute.

This whole process is called ***building an exectuable***.

Recall **.c** refers to C files and not C++ files.

Stick to vim little boy.

1.2.2 Programming Your First C++ Application

Note all code will be in ../code dir.

Listing 1.1 Hello.cpp, the Hello World Program

Using linux, you can create the executable via

```
g++ -o hello Hello.cpp
```

This command instructs g++ to create an **exectuable** named `hello`

clang equivalent command:

```
clang++ -o hello Hello.cpp
```

Both g++ and clang++ support many of the same options or ***flags***, for example:

```
g++ -std=c++20 -Wall -Wextra -O2 -o hello Hello.cpp
clang++ -std=c++20 -Wall -Wextra -O2 -o hello Hello.cpp
```

- `-std=c++20`: Compiles the program using the C++20 language standard. Other common standards include `c++11`, `c++14`, `c++17`, and `c++23`.
- `-Wall`: Enables a broad set of commonly useful compiler warnings. While it does not enable *all* warnings, it catches many common programming mistakes.
- `-Wextra`: Enables additional warnings beyond those provided by `-Wall`. Using both flags together is considered good practice during development.
- `-O2`: Enables a moderate level of compiler optimizations to improve the performance of the generated executable while keeping compilation times reasonable. Other optimization levels include:
  - `-O0`: No optimization (default for debugging).
  - `-O1`: Basic optimizations.
  - `-O2`: Recommended general-purpose optimization.
  - `-O3`: More aggressive optimizations that may increase executable size.
  - `-Ofast`: Enables aggressive optimizations that may sacrifice strict standards compliance.
- `-g`: Includes debugging information in the executable so that a debugger (such as `gdb` or `lldb`) can display source code lines, variables, stack frames, and other useful debugging information. This flag does *not* make the program slower by itself; it simply embeds additional symbolic information.
- `-o hello`: Specifies the name of the output executable. Without this flag, the default executable name is typically `a.out` on Linux/macOS or `a.exe` on Windows (depending on the compiler).

Some more feature of compilers include ***Preprocessor Definitions***

The `-D` compiler flag defines a preprocessor macro at compile time. For example,

```
g++ -DDEBUG -o hello Hello.cpp
```

is equivalent to having

```
#define DEBUG
```

before preprocessing begins. This allows for *conditional compilation*:

```
#ifdef DEBUG
    std::cout << "Debug mode enabled\n";
#endif
```

The code inside the `#ifdef DEBUG` block is only compiled when the `DEBUG` macro is defined. This technique is commonly used to enable debugging, logging, assertions, or other development-only features.

Remember to actually execute the executable by calling it like ./hello on Linux or hello.exe on Windows.

A thing to remember is through the evolution of C++, the same program can be compiled using different compilers on different operating systems.

1.3 QA

- **Q: Can I ignore warning messages from my compiler?**

  A: These warnings show to the programmer a better/safer way to go about things.
  It is up to them to decide to ignore or not. Generally, it's best to address them for production level code.

- **Q: How does an interpreted language differ from a compiled language?**

  A: Languages such as Python or Javascript are **interpreted** (for the most part). There is **no compilation step**.

  An ***interpreted language*** use an **interpreter** that directly reads the script text file (code) and performs the desired actions.

  Consequently, you need to have the interpreter installed on a machine where the script needs to be executed; consequently, **performance usually takes a hit** as the interpreter works as a **runtime translator** between the microprocessor and the code written.

- **Q: What are runtime errors, and how are they different from compile-time errors?**

  A: Errors that happen when you execute your application are called ***runtime errors***.

  Compile-time errors do not reach the end user and are typically caused by **syntax** (e.g., missing semicolons, unmatched braces) or **semantic** (e.g., undeclared variables, type mismatches) errors. They prevent the compiler from successfully generating an executable.

1.4 Quiz

1. What is the difference betwwen an interpreter and compiler?

   A: An **interpreter** is a tool that interprets what you code (or an **intermediate byte code**) and performs certain actions.

   A **compiler** takes your code as an input and generates an **object file**. In the case of C++, after **compiling** and **linking** have have an **executable** that can run directly by the processor without need for any further interpretation.

2. What does the linker do?

   A: A compiler takes a C++ file as input and generates an **object file** in machine language.

   Often your code has **dependencies** on libraries and functions in other code files. Creating these **links** and generating an executable that integrates all dependencies is the job of the **linker**.

3. What are the stages in the normal development cycle?

   A: Code. Compile to create object file. Link to create executable. Execute. Debug. Fix any errors. Repeat.

   In many/most cases compiling and linking are one step.

1.5 Exercies

1. E1
2. E3

2 The Anatomy of a C++ Program

C++ programs are comprised into ***classes*** comprising ***member functions*** and ***member variables/fields***.

2.1 Parts of the Hello World Program

Analyze ts:

Listing 2.1 HellowWorldAnalysis.cpp: Analyze a Simple C++ Program.

The C++ program can be classified into two parts:

1. the ***preprocessor directives*** that start with a # and
2. the ***main body*** of the program that starts with `int main()`.

2.2 Preprocessor Directive #include

The ***preprocessor*** is a tool that runs before the actual **compilation** starts.
Preprocessor directives are commands to the **preprocessor** and always start with a # symbol.

```
#include <iostream>
#define PI 3.14159
#ifdef DEBUG
#endif
#ifndef HEADER_H
#endif
```

In our code Listing 2.1, the `#include <filename>` tells the preprocessor to take the contents of the file (`iostream` in our example) and inc9ulude them at the line where the directive is made.

`iostream` is a ***standard header file*** that enables the usage of `std::cout` to display "Hello World" on the screen.

The compiler was able to compile the code that contains `std::cout` b/c we instructed the preprocessor to include the definition of `std::cout`

Note: We can include our own header files using `#include "relative/path/to/FileB.h"`.
Use **double quotes** (`""`) for project headers and **angle brackets** (`<>`) for standard library headers (e.g., `<iostream>`, `<vector>`).

The underlying rule is:
#include "..." $\rightarrow$ searches the current/project directories first, then system include paths.
#include <...> $\rightarrow$ searches only the system include paths.

The syntax ("" vs <>) does not define "own vs standard".
The include search path (set with -I) is what actually lets the compiler find your own headers in different folders.

2.3 The Body of Your Program main()

Following the **preprocessor directive(s)** is the ***body*** of the program characterized by the function **main()**.

The execution of a C++ program always starts here. It is a standardized convention that **main()** is of type **int**, which stands for **integer**.

int main() can take many forms in C/C++:

```
int main() {
    return 0;
}
```

Standard C++ entry point with no command-line arguments (also valid in C, though `int main(void)` is preferred).

```
int main()
{
    return 0;
}
```

Same as above; only the brace style differs.

```
int main(void)
{
    return 0;
}
```

Standard C entry point explicitly specifying that no arguments are accepted.

```
int main(int argc, char *argv[])
{
    return 0;
}
```

Standard entry point that receives command-line arguments.

```
int main(int argc, char **argv)
{
    return 0;
}
```

Equivalent to the previous form since array parameters decay to pointers.

```
int main(int argc, char *argv[], char *envp[])
{
    return 0;
}
```

Implementation-defined form that also receives environment variables.

```
int wmain(int argc, wchar_t *argv[])
{
    return 0;
}
```

Microsoft-specific Unicode entry point using wide-character command-line arguments.

```
void main()
{
    /* Non-standard */
}
```

Accepted by some compilers but not part of the C or C++ standards.

2.4 Returning a Value

Functions must have a **return type**. If a function's return type is `void`, it does not return a value, so you do not need to write a `return` statement (although you may write `return;` to exit the function early).

main() is a function of type **int**, so it would always return an **int** or integer.

main() having a return type of type int can be useful for OSes to determine the state of which the program had exited. For example, in many cases one application is launched by another and the ***parent application*** (that launches it) wants to know if the ***child application*** (that was launched) has completed its task successfully.

The programmer can use the **return value** of **main()** to convery a success or error state to the **parent application**.

Conventionally, **main()** returns:

- **0** for **success**
- **−1** for **error**

However, the return type is an **integer** and you can still have flexibility to convey many different states of success or failure using the available range of integer return values.

Side note: C/C++ is case-sensitive, so Int and int aren't the same. Int would have to be a class/struct/typedef/etc and wouldn't be the same as the standard int type.

2.5 Namespaces

The reason why you used `std::cout` and not just `cout` is that the ***artifact*** (cout) is apart of the ***standard (std) namespace***.

So, what are **namespaces**?

First, assume that you didn't use the **namespace qualifier** in invoking `cout` and assume `cout` existed in **two locations** known to the **compiler** — which one should the compiler invoke?

This trivially causes a conflict and the **compilation fails**.

Namespaces to the rescue. Namespaces are **names** given to parts of code that reduc the potential for **naming conflicts**.

By invoking **std::cout**, you are telling the compiler to use that one unique `cout` that is available in the **std namespace**.

To use the namespace you simply do:

```
using namespace <namespace_name>;

// like

using namespace std;
```

To create a **namespace**, you do:

```
namespace MyNamespace
{
    int x = 10;

    void hello()
    {
        std::cout << "Hello!\n";
    }
}
```

You access members using the scope resolution operator ::

```
int main()
{
    std::cout << MyNamespace::x << '\n';
    MyNamespace::hello();
}
```

Instead of writing `<namespace_name>::whatever` every time, you can write:

```
using namespace MyNamespace;
```

to make all names in `MyNamespace` directly accessible.
Alternatively, if you only want to bring a single name into scope, use:

```
using MyNamespace::whatever;
```

std stands for the **standard** namespace and you invokve functions, etc there that have been **ratified** by the ISO Standards Committee.

Listing 2.2 The using namespace Declaration

Listing 2.3 Another Demo of the *using* keyword

However, with all this said:
Avoid using namespace ...; in global scope, especially in headers and large programs. this limits damage/errors to a smaller scope.

- The difference btw `using namespace std` and `using std::cout or std::endl` is that the **former** allows **all artifacts** in the **std namespace** (cout, cin, etc) to be used without explicit inclusiong of the namespace.
- With the **latter**, the namespace would explicitly be restricted to only **std::cout** and **std::endl**.

2.6 Functions in C++

- We've already written a function before: **int main()**.
- Functions have a **return type** like **int** for **int main()**.
- Other return types include:
  - **void** — the function returns no value.
  - **float** — returns a single-precision floating-point number.
  - **double** — returns a double-precision floating-point number.
  - **char** — returns a single character.
  - **bool** — returns a boolean value (true/false).
  - **std::string** — returns a string object (C++).
  - **int* / double* / etc.** — returns a pointer to a value.
  - **custom types (struct/class)** — returns user-defined data types.
- We also have ***lambda functions***, which are anonymous functions (they do not have an explicit name).
- **Listing 2.4**, Declaring, Defining, and Calling a Function

- ***Function declarations*** are, for example:

```
int DemoConsoleOutput();
```

  This is a function declaration without a body (i.e. no definition). It informs the compiler that a function with this name, return type, and parameter list exists somewhere in the program.

  This allows the function to be called before its actual definition appears in the file. The **compiler** assumes the **definition** will be provided later (possibly in the same file or in another translation unit).

- ***Function definitions*** provide the actual implementation of the function. For example:

```
int DemoConsoleOutput()
{
    std::cout << "Hello\n";
    return 0;
}
```

  A function definition includes the function body, where the actual logic is written and executed when the function is called.

  Function definitions allocate the actual code that runs at runtime, whereas declarations only describe the function's interface.

  Using separate declarations and definitions helps with:
  - better code organization (especially in large projects),
  - separation of interface (.h files) and implementation (.cpp files),
  - allowing functions to be used across multiple files.

- **Listing 2.5** Using the Return Value of a Function
- Functions
  - can take **parameters** (values defined in the function signature that receive input from the caller as **arguments**),
  - can be **recursive** (a function that calls itself and requires a base case to stop),
  - can contain multiple return statements (though if not inside a condition, the compiler may issue a warning),
  - can be ***overloaded*** (multiple functions with the same name but different parameter lists),
  - can be **overwritten** (more correctly: overridden in inheritance, where a derived class replaces a base class function),
  - can be **expanded in-line** by the compiler (the compiler replaces the function call with the function body to reduce call overhead),
  - and much more.
- This is discussed in **Lesson 7**.

2.7 Basic Input Using std::cin and Output using std::cout

- We use the **console** to write and read information.

- use **std::cout** (standard "see-out") to **write** text to the console.

  std::cout is accompanied by $<<$

  ```
  std::cout << "Hello World" << std::endl;
  ```

- use **std::cin** (standard "see-in") to **read** text from the console.

  std::cin is accompanied by $>>$

  ```
  std::cin >> Variable;
    std::cin >> Variable1 >> Variable2;
  ```

- **Listing 2.6** Use cin and cout

2.8 QA

- **Q: What does #include do**?

  A: This is a **directive** to the **preprocessor** when you call your compiler. This specific directive causes the contents of the file named in <> after #include to be **inserted** at that line as if it were typed at that location in your source code.

- **Q: When do you need to program command-line arguments?**

  A: To supply **options** that allow the user to alter the behavior of a program.

  For example, `ls` in Linux or `dir` in Windows enables you to see the contents within the current directory or folder. To view files in another directory, you specify the path of the same using command-line arguments, as in `ls /` or `dir \`.

2.9 Quiz

- **Q: What is the problem in declaring `Int main()`?**
  A: `int main()` is what it must be.

- **Q: Can comments be longer than one line**?
  A: Yes via `/* */`

2.10 Exercises

1. e1.cpp

3 Using Variables, Declaring Constants

- ***Variables*** are points in memory that store data in a finite amount of time.
- ***Constants*** are points in memory that define **artifacts** thare are **not allowed** to **change**.

3.1 What Is a Variable?

3.1.1 Memory and Addressing

- All computers contain a ***microprocessor*** and a certain amount of **memory** for **temporary storage** called ***Random Access Memory (RAM)***.
- In addition, many devices also allow for data to be **persisted** on a ***storage device*** such as the **hard disk**.
- The **microprocessor** executes your application, and in doing so works with the **RAM** to fetch the application **binary code** to be executed as well as the **data** associated with it.

- **RAM** can be thought as a **storage** akin to a row of lockers in a hallway, each locker having a **number** — that is, an ***address***.
- To access a location in **memory**, say location 578, the processor needs to be asked via an **instruction** to **read** a value from there or **write** a value to it.
- There exist ***virtual addresses***, which allow each process to have its **own address space** independent of **physical memory**. These **virtual addresses** are translated into **physical addresses** by the operating system and hardware.
- The main purpose of **virtual addresses** is to provide **memory isolation**, **memory protection**, and the illusion that each process has its own large, contiguous block of memory, even though **physical memory is shared** among many processes.

3.1.2 Declaring Variables to Access and Use Memory

- For variables, it's best to initialize it no matter if you use it, so that you know what kind of value it holds.
- If you don't, you wouldn't know the value, which may cause problems ahead.
- **Listing 3.1**
- The compiler does its job of mapping the variable to a location **in memory** and takes care of the associated **memory-address** bookkepping for you.
- The programmer works with the human-friendly names, while the **compiler** manages **memory-addressing** and creates the instructions for the microprocessor to execute in working with the **RAM**.

3.1.3 Declaring and Initializing Multiple Variables of a Type

- You can condense the declaration of variables of the same type to one line of code:

```
int firstNum = 0, secondNum = 0, thirdNum = 0; // and so on
```

- Note: data stored in variables is data stored in RAM. It is not persistent and is lost upon application termination. To persist it, as of today, you must write it to a file on disk.

3.1.4 Scope of a Variable

- When outside the ***scope*** of a variable, the **variable names** will not be recognized by the compiler and your program won't compile.
- Beyond its scope a variable is an **unidentified entity** that the compiler knows nothing of.
- **Listing 3.2** Scope

3.1.5 Global Variables

- Not best practice for production projects in source files.
- All functions within the source file can access global variables declared in that file (provided they are in scope). Functions in other source files cannot access them unless the variables have external linkage (e.g., are declared with `extern` in a header and defined in exactly one source file).

  Example:

  ```
  // globals.h
  extern int g_value;

  // globals.cpp
  int g_value = 42;
  ```

- **Listing 3.3** Global variables

3.2 Common Compiler-Supported C++ Variable Types

- `bool` – stores `true` or `false`.
- `char` – typically 1 byte, range: -128 to 127 (or 0 to 255 if unsigned).
- `signed char` – 1 byte, range: -128 to 127.
- `unsigned char` – 1 byte, range: 0 to 255.
- `short` (or `short int`) – typically 2 bytes, range: $-32{,}768$ to $32{,}767$.
- `unsigned short` – typically 2 bytes, range: 0 to $65{,}535$.
- `int` – typically 4 bytes, range: $-2{,}147{,}483{,}648$ to $2{,}147{,}483{,}647$.
- `unsigned int` – typically 4 bytes, range: 0 to $4{,}294{,}967{,}295$.
- `long` – typically 8 bytes on Linux/macOS and 4 bytes on Windows.
- `unsigned long` – unsigned version of `long`.
- `long long` – typically 8 bytes, range: -9.22×10^{18} to 9.22×10^{18}.
- `unsigned long long` – typically 8 bytes, range: 0 to 1.84×10^{19}.
- `float` – typically 4 bytes, approximately $\pm 3.4 \times 10^{38}$ (about 7 decimal digits of precision).
- `double` – typically 8 bytes, approximately $\pm 1.7 \times 10^{308}$ (about 15–16 decimal digits of precision).
- `long double` – extended precision floating-point type (size and precision are implementation-defined).
- `void` – represents the absence of a value (e.g., functions that return nothing).

Fixed-Width Integer Types (`<cstdint>`)

- `int8_t` – exactly 8 bits, range: -128 to 127.
- `uint8_t` – exactly 8 bits, range: 0 to 255.
- `int16_t` – exactly 16 bits, range: $-32{,}768$ to $32{,}767$.
- `uint16_t` – exactly 16 bits, range: 0 to $65{,}535$.
- `int32_t` – exactly 32 bits, range: $-2{,}147{,}483{,}648$ to $2{,}147{,}483{,}647$.
- `uint32_t` – exactly 32 bits, range: 0 to $4{,}294{,}967{,}295$.
- `int64_t` – exactly 64 bits, range: $-9{,}223{,}372{,}036{,}854{,}775{,}808$ to $9{,}223{,}372{,}036{,}854{,}775{,}807$.
- `uint64_t` – exactly 64 bits, range: 0 to $18{,}446{,}744{,}073{,}709{,}551{,}615$.

3.2.1 Signed vs Unsigned integers

- ***Sign*** implies **positive** or **negative**.
- All numbers you work with using a computer are stored in the **memory** in the form of ***bits*** and ***bytes***.
- A **memory location** that is **1 byte** long contains **8 bits**.
- Each **bit** can either be **0** or **1**.
- Thus, a **memory location** that is **1 byte** long can contain a maximum of **2** to the power of **8 values** — that is **256** unique values.
- Similarly, a **memory location** that is **16 bits** large (or **2 bytes**) can contain **2** to the power of **16 values** — that is, **65, 536** unique values.
- Think of it as just

$$2^{\text{number of bits}} = \text{quantity of how many numbers we can represent}$$

- If the values are ***unsigned*** (i.e., only non-negative), then an n**-bit** integer can represent values from 0 to $2^n - 1$.
- For example:
  - **1 byte** (8 bits): 0 to $2^8 - 1 = 255$.
  - **2 bytes** (16 bits): 0 to $2^{16} - 1 = 65{,}535$.
- For an 8-bit unsigned integer, the place values are:

$$2^7 \quad 2^6 \quad 2^5 \quad 2^4 \quad 2^3 \quad 2^2 \quad 2^1 \quad 2^0$$

  If every bit is set to 1,

$$11111111_2 = 2^7 + 2^6 + \cdots + 2^1 + 2^0 = 2^8 - 1 = 255.$$

- To represent **negative numbers**, signed integers are typically stored using ***two's complement***. Rather than sacrificing the leftmost bit, it is assigned a weight of -2^{n-1}, while all remaining bits retain their usual positive weights.
- For an n**-bit signed integer**, the representable range is

$$-2^{n-1} \text{ to } 2^{n-1} - 1.$$

- Thus, an **8-bit signed integer** has the range

$$-2^7 \text{ to } 2^7 - 1 = -128 \text{ to } 127.$$

- Since there are $2^8 = 256$ unique bit patterns, exactly half represent negative values and half represent non-negative values.

- The place values for an 8-bit signed integer are

$$-2^7 \quad 2^6 \quad 2^5 \quad 2^4 \quad 2^3 \quad 2^2 \quad 2^1 \quad 2^0.$$

  Therefore,

$$\texttt{10000000} = -2^7 = -128,$$

  and

$$\texttt{11111111} = -128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = -1.$$

- Recoginize the smallest/minimum is when only the **MSB** is 1 and the rest are 0, which gives our msb weight of -2^{n-1}.

3.2.2 Signed integer types: short, int, long, and long long

- These are the fundamental **signed integer types** in C++, used to store whole numbers (both positive and negative).
- They primarily differ in the amount of memory they occupy and, consequently, the range of values they can represent.
- In general, `short` stores smaller integers than `int`, while `long` and `long long` can store progressively larger values.
- The exact size of each type is **implementation-defined** (depends on the compiler and platform), although modern systems typically use:
  - `short`: 2 bytes
  - `int`: 4 bytes
  - `long`: 4 bytes (Windows) or 8 bytes (Linux/macOS)
  - `long long`: 8 bytes
- When an exact integer size is required (e.g., exactly 32 or 64 bits), prefer the fixed-width integer types from `<cstdint>` such as `int32_t` and `int64_t`.

3.2.3 Unsigned integer types: unsigned short, unsigned int, unsigned long, unsigned long long

- These are the fundamental **unsigned integer types** in C++, used to store only **non-negative** whole numbers.
- Since no bits are reserved for representing negative values, unsigned integers can represent approximately twice the positive range of their signed counterparts of the same size.
- They have the same typical sizes as their signed versions:
  - `unsigned short`: 2 bytes
  - `unsigned int`: 4 bytes
  - `unsigned long`: 4 bytes (Windows) or 8 bytes (Linux/macOS)
  - `unsigned long long`: 8 bytes
- An unsigned n**-bit** integer has the range

$$0 \text{ to } 2^n - 1.$$

- Unsigned integers are useful when negative values are impossible (e.g., sizes, counts, and bit manipulation), but care should be taken when mixing signed and unsigned types, as implicit conversions can produce unexpected results.

3.2.4 Integer Overflow

- ***Integer overflow*** occurs when the result of an arithmetic operation is outside the range that can be represented by the integer's type.
- For example, an 8-bit signed integer can only store values from -128 to 127. Attempting to compute
$$127 + 1$$
produces a value that cannot be represented.
- For **unsigned integers**, overflow is **well-defined** in C++. Arithmetic wraps around modulo 2^n, where n is the number of bits. For example, an 8-bit unsigned integer satisfies
$$255 + 1 = 0.$$
- For **signed integers**, overflow results in ***undefined behavior***. This means the C++ standard does not specify what should happen—the program may appear to wrap around, produce an unexpected value, or behave unpredictably.
- To avoid overflow, choose an integer type with a sufficiently large range (e.g., `int64_t` instead of `int32_t`) or check whether an operation would exceed the type's limits before performing it.

- **Listing 3.4** Signed and Unsigned overflow
- The output shows the wrap around effect of incrementing and overflowing for both types.

3.2.5 Floating-Point types `float` and `double`

- ***Floating-point types*** are used to store **real numbers** (numbers with a fractional or decimal component), such as `3.14` or `-0.001`.
- The two most commonly used floating-point types are:
  - `float` – typically occupies **4 bytes** and provides about **7 decimal digits** of precision.
  - `double` – typically occupies **8 bytes** and provides about **15–16 decimal digits** of precision.
- Floating-point numbers are generally represented according to the ***IEEE 754*** standard, which stores a number using three components:
  - a **sign bit**,
  - an **exponent**,
  - and a **significand (mantissa)**.
- Since only a finite number of bits are available, many decimal values (such as `0.1`) cannot be represented exactly. As a result, floating-point arithmetic may introduce small rounding errors.
- Floating-point types should not be used when exact decimal precision is required (e.g., financial calculations). Instead, use an appropriate decimal or fixed-point representation.
- In general, prefer `double` over `float` unless reduced memory usage or compatibility with specific hardware is important.

- Under the hood, `float` and `double` are stored using the ***IEEE 754 floating-point format***. A number is represented in **scientific notation in base 2**:

$$(-1)^{\text{sign}} \times 1.\text{fraction} \times 2^{\text{exponent}}$$

- A `float` (32 bits) is split into:
  - 1 bit for the **sign**
  - 8 bits for the **exponent**
  - 23 bits for the **fraction (mantissa)**
- A `double` (64 bits) is split into:
  - 1 bit for the **sign**
  - 11 bits for the **exponent**
  - 52 bits for the **fraction (mantissa)**
- For example, the value 5.75 is stored as:

$$5.75_{10} = 101.11_2 = 1.0111 \times 2^2$$

  So the stored components are:
  - sign = 0 (positive)
  - exponent = 2 (stored with a bias)
  - fraction = 0111 (remaining bits after the leading 1)

For the above example: 5.75

- Whole part: 5

$$5 = 101_2$$

- Fractional part: 0.75
  - $0.75 \times 2 = 1.5 \rightarrow 1$
  - $0.5 \times 2 = 1.0 \rightarrow 1$

$$0.75 = 0.11_2$$

- Combine whole and fraction:

$$5.75 = 101.11_2$$

- Normalize (move binary point left 2 places):

$$101.11_2 = 1.0111 \times 2^2$$

- **Why we multiply by 2 repeatedly for the 2nd step?**
  We are converting a decimal fraction (base 10) into a binary fraction (base 2).
  In binary, each fractional bit represents a negative power of two:

$$\frac{1}{2}, \quad \frac{1}{4}, \quad \frac{1}{8}, \quad \frac{1}{16}, \ldots$$

  We therefore need a method to determine which of these powers of two are present in the number.
  Multiplying by 2 repeatedly allows us to extract these bits one at a time by shifting the fraction:
  - the integer part after multiplication gives the next binary digit,
  - the remaining fractional part is used for the next step.

Converting back to base 10:
We start from the binary result:

$$5.75 = 101.11_2$$

Split into integer and fractional parts:

$$101.11_2 = 101_2 + 0.11_2$$

Convert each part separately:

- Integer part:

$$101_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 4 + 0 + 1 = 5$$

- Fractional part:

$$0.11_2 = 1 \cdot 2^{-1} + 1 \cdot 2^{-2} = \frac{1}{2} + \frac{1}{4} = 0.75$$

Therefore:

$$101.11_2 = 5.75_{10}$$

3.3 Size of a Variable using sizeof

- **Size** is the amount of **memory** that the compiler reserves when a variable is declared.
- The size of a variable depends on its **type**.
- We can see the size via `sizeof(<type>)` and this outputs the **size of type in bytes**.
- **Listing 3.5** Finding the Size of Standard C++ Variable Types
- Note these sizes are **dependent** on the 32 or 64-bit **compiler** (not necessarily an OS).

3.4 Automatic Type Inference Using auto

- Compilers supporting C++11 and beyond give the option of **not** having to **explicitly** specify the variable type when using the keyword ***auto***

```
auto coinFlippedHeads = true;
```

- We leave the task of defining an exact type for variable `coinFlippedHeads` to the **compiler**.
- The **compiler** checks the nature of the value the variable is initialized to and then decides the best possible type that suits this variable.
- In the above example, the compiler determines `bool` as the type that suits variabled `coinFlippedHeads` and **internally** treats `coinFlippedHeads` as a `bool`.
- **Listing 3.6** Using auto
- When you **don't initialize** a variable of type **auto**, you get a **compile error** like:

```
error: declaration of ‘auto willGetError’ has no initializer
    9 |         auto willGetError;
```

3.5 typdef and/or using to Substitute a Variable's Type

- You can subsitute a variables types to something that you might find convenient.
- You would use the keyword ***typedef*** for that.
- Here is an example where a programmer wants to call an **unsigned int** a descriptive `STRICTLY_POSITIVE_INTEGER`.

```
typedef unsigned int STRICTLY_POSITIVE_INTEGER
STRICTLY_POSITIVE_INTEGER numEggsInBasket = 4532;
```

- Note, an equivalent keyword now in practice in modern C++ (introducd in C++11) is the keyword ***using***:

```
// Modern C++ equivalent
using STRICTLY_POSITIVE_INTEGER = unsigned int;
STRICTLY_POSITIVE_INTEGER numEggsInBasket = 4532;
```

- When compiled, the first line tells the compiler that a `STRICTLY_POSITIVE_INTEGER` is an **unsigned int**. At later stages when the compiler encounters the already defined type `STRICTLY_POSITIVE_INTEGER`, it substitutes it for **unsigned int** and continues compilation.
- **typedef** or **type substitution** is useful for when dealing with complex types, for example types that use **templates**.

3.6 What is a Constant?

- C++ enables you to define a variable as **constant** that means it **cannot change (its value)** after **declaration**.
- Assignment to a constant cause compilation errors.
- Constants can be:
  - Literal constants
  - Declared constants using the `const` keyword
  - Constant expressions using the `constexpr` keyword (new since C++11)
  - Enumerated constants using the `enum` keyword
  - Defined constants that are not recommended and deprecated.

3.6.1 Literal constants

- **Literal constants** can be of many types — **integer, string, and so on**.
- In our Hello World program, here "Hello World" is a ***string literal constant*** (or just **string constant**) meaning they cannot be modified.

  String literals are **immutable** in C++. Attempting to modify a string literal results in **undefined behavior**. For example:

```
const char* str = "Hello";
str[0] = 'J';   // Error: cannot modify a string literal

char* ptr = "Hello";
ptr[0] = 'J';   // Undefined behavior
```

  If you need a modifiable sequence of characters, initialize a character array instead:

```
char str[] = "Hello";
str[0] = 'J';   // OK

std::cout << str;   // Prints: Jello
```

  Here, the compiler creates a writable character array by copying the contents of the string literal.

  Note: for `const char* str = "Hello";` the pointer itself is *not* placed in read-only memory—the **string literal** is.

  For example:

```
const char* str = "Hello";
```

  In this declaration:

  - `"Hello"` is a ***string literal***, and its characters are typically stored in read-only memory.
  - `str` is a pointer variable (usually stored on the stack if it is a local variable). It stores the address of the first character of the string literal.

– The `const` qualifier prevents the characters from being modified through `str`.

In contrast,

```
char str[] = "Hello";
```

creates a writable character array by copying the contents of the string literal into the array. Since the array is a separate copy, modifying it is valid:

```
str[0] = 'J';   // OK
```

- For ***int literals***, we have also been using them all along in our programs. An integer literal such as `42` or `0` is simply a fixed value written directly in the source code.

  Unlike variables, an integer literal is not a named object in memory that we can modify. Instead, it is used by the compiler as a constant value when evaluating expressions or initializing variables.

  For example:

  ```
  int x = 42;
  x = x + 1;
  ```

  Here, `42` is an integer literal that is copied into the variable `x`. We are not modifying the literal itself; we are modifying the variable `x` that stores a copy of that value.

- This is why the following declaration makes sense:

  ```
  const char* p = "Hello";
  ```

  The string literal `"Hello"` corresponds to a real array of characters stored in memory (typically in a read-only section of the program). Therefore, `p` can hold the address of its first character.

  In contrast, the following is invalid:

  ```
  int* p = &42;   // invalid
  ```

  This does not work because `42` is an integer literal, not a memory object. It is a value used directly by the compiler, and it does not have a valid, addressable memory location. Therefore, taking its address using `&` is not allowed.

- ***Binary literals*** were introduced in **C++14**. They allow writing numbers in base 2 using the prefix `0b` or `0B`. For example:

  ```
  int x = 0b1010;   // 10 in decimal
  ```

- ***Octal literals*** have existed since early C++. They use a leading zero `0`. For example:

  ```
  int x = 075;   // 61 in decimal
  ```

  Here, `075` is interpreted as an octal (base-8) number rather than a decimal.

- ***User-defined literals*** were introduced in **C++11**. They allow programmers to define custom suffixes for literals, enabling more readable and expressive code.

  For example, we can define a literal to represent distances in meters:

  ```
  long double operator"" _m(long double x) {
      return x;
  }

  long double distance = 5.0_m;
  ```

  Here, `5.0_m` is a user-defined literal, where the suffix `_m` converts the number into a meaningful unit.

  User-defined literals can be used with integers, floating-point numbers, characters, and strings, and they are commonly used in libraries to improve readability (e.g., time durations, units, or complex numbers).

3.6.2 Declaring Variables as Constants Using const

- `const` is meant to be used **before** the variable type is declared.

```
const type-name constant-name = value;
```

- **Listing 3.7** Declaring a Constant Called pi
- Here is the compile-error if you try to modify a const

```
const double pi = 22.0 / 7;
pi = 345;

error: assignment of read-only variable ‘pi’
   11 |          pi = 345;
```

- It's good programming practice to define variables that are **not supposed to change** as `const` since it shows the programmer has thought about ensuring the correctness and constantness of data (acts as protection).

3.6.3 Constant Expression Using constexpr

- Keyword ***constexpr*** allows function-like declartions of **constants**:

```
constexpr double GetPi() {return 22.0 / 7;}
```

- One **constexpr** can use another:

```
constexpr double TwicePi() {return 2 * GetPi();}
```

- `constexpr` make look like a **function**, however, allows for optimization possibilities from the compiler's view. So long as a compiler is capable of evaluting a **constant expression** to a **constant**, it can be used in statements and expressions at places where a constant is expected.

- In the preceding example, `TwicePi()` is a **constexpr** that uses a **constant expression** `GetPi()`. This will possibly trigger a **compile-time optimization** wherein every usage of `TwicePi()` is simply **replaced** by 6.28571 by the compiler, and **not** the code that would calculate $2 \times 22/7$ when executed.

- We can also combine ***constexpr*** with ***const*** when we want to emphasize that a value is both **compile-time computable** and **read-only after initialization**. While `constexpr` already implies constness for the expression, adding `const` can still be useful for clarity in variable declarations.

  For example:

```
constexpr double GetPi() { return 22.0 / 7; }

const double pi1 = GetPi();          // runtime or compile-time constant
constexpr double pi2 = GetPi();      // guaranteed compile-time constant
```

  Here, `pi2` must be evaluated at compile time, while `pi1` is simply a read-only value initialized from a constant expression. In practice, `constexpr` is preferred when compile-time evaluation is desired, since it already enforces immutability and constant-expression rules.

- We can also place ***const*** at the end of a member function declaration to indicate that the function does not modify the state of the object. This is different from `const` variables or `constexpr` functions.

  For example:

```
class Circle {
public:
    double GetRadius() const {
        return radius;
    }

    void SetRadius(double r) {
        radius = r;
    }

private:
    double radius;
};
```

Here, `GetRadius() const` guarantees that no data members of the object will be modified inside the function. If we attempt to modify any member variable inside a `const` function, the compiler will produce an error.

This `const` qualifier at the end of a function is commonly used for **getter functions** to ensure they are safe and do not change object state.

- **Listing 3.8** using constexpr

- The two **constexpr**'s may look like functions, but they are not exactly.

  Functions are invoked at program **execution time**.

  Constant expressions are already substituted to their hard values by the compiler at **compile time**.

- So here in the listing, the `constexpr` saves **time and computation** to the same calculation being contained in a function.

- Note if the block/body/defintion of the **constexpr** relies on user input/numerics not known at compile time, then the compiler treats the **constexpr** as a **regular function**, i.e. no compile-time optimization.

3.6.4 Enumerations

- Sometimes you want a **variable** whose values are **restricted** to a certain **set** of values defined by you.
- ***Enumerations*** are the tool and are characterized by the *keyword* ***enum***.
- Example:

```
enum RaindbowColor
{
 Violet = 0,
 Indigo,
 Blue,
 Green,
 Yellow,
 Orange,
 Red
};
```

- A **cardinal direction example**:

```
enum CardinalDirections
{
 North,
 South,
 East,
 West
};
```

- **Enumerations** are **user-defined** types.
- Variables of this **type** can be assigned to a **range of values** restricted to the **enumerators** contained in the **enumeration**.
- So, if defined a variable that contains the colors of a rainbow, you declare the variable like this:

```
RainbowColors MyFavColor = Blue;
```

- This works for a traditional (unscoped) enumeration because the enumerator names (such as `Blue`) are placed directly into the surrounding scope.

  If the enumeration were declared as a `enum class` (a scoped enumeration, introduced in C++11), the enumerators would remain inside the scope of the enumeration. In that case, we would need to qualify the name:

```
RainbowColors MyFavColor = RainbowColors::Blue;
```

  Scoped enumerations help prevent name collisions by requiring the enumeration name when referring to its values.

- C++ provides two kinds of enumerations: the traditional ***enum*** and the newer ***enum class*** (introduced in **C++11**). The main difference is that an `enum class` is **scoped**, meaning its enumerator names remain inside the enumeration and do not pollute the surrounding scope.

  For example, with a traditional enumeration:

```
enum RainbowColors { Red, Orange, Yellow, Green, Blue };

RainbowColors color = Blue;   // OK
```

The name `Blue` is placed directly into the surrounding scope, so it can be used without qualification. However, this can lead to name conflicts if another enumeration also contains an enumerator named `Blue`.

With a scoped enumeration:

```
enum class RainbowColors { Red, Orange, Yellow, Green, Blue };

RainbowColors color = RainbowColors::Blue;   // OK
```

Here, the enumerator must be qualified with the enumeration name (`RainbowColors::Blue`). This avoids name collisions and provides better type safety, making `enum class` the preferred choice in modern C++.

- Generally, the compiler converts the **enumerator** such as *Violet* and so on into integers.

  Each **enumerated value** specified is **one more than** the **previous value** if not explicitly initialized.

  You have the choice of specifying a **starting value**, and if this is **not specified**, the compiler takes it as **0**.

- You can still specify the explicit value against each of the enumerated constants by initializing them.

```
enum ErrorCode
{
    Success = 0,
    Warning = 100,
    Error = 200
};
```

- **Listing 3.9** Enumerated Values

3.6.5 Defining Constants Using #define

- For C++ this kind of defining is not recommended.
- This is legacy support (C defines → OK in C but we're in C++).
- We can defned pi like before as

```
#define pi 3.14286
```

- **#define** is a ***preprocessor macro***.
- What is done here is that all mentions of `'pi'` henceworth are replaced by 3.14286 for the **compiler** to process.
- Note: the compiler neither knows nor cares about the **actual type** of the **constant** in question (it's just a text replacement (read: non-intelligent replacement) done by the **preprocessor**).

3.7 Keywords that Cannot be Used as Variable or Constant Names

The following are examples of **reserved keywords** in C++.
Since these words have special meanings to the compiler, they cannot be used as identifiers (such as variable, constant, or function names).

- asm
- auto
- bool
- break
- case
- catch
- char
- class
- const
- constexpr
- continue
- default
- delete
- do
- double
- else
- enum
- explicit
- export
- extern
- false
- float
- for
- friend
- goto
- if
- inline
- int
- long
- mutable
- namespace
- new
- nullptr
- operator
- private
- protected
- public
- register
- return
- short
- signed
- sizeof
- static
- struct
- switch
- template
- this
- throw
- true
- try
- typedef
- typeid
- typename
- union
- unsigned
- using
- virtual
- void
- volatile
- while

3.8 QA

- **Q: Why define constants at all if you can use regular variables instead of them?**

  A: **Constants**, especially those declared using the keyword `const`, are your way of telling the **compiler** that the **value** of a particular variable be **fixed** and **not allowed** to **change**.

  It ensures/enforces correctness and constantness and quality of your code, so that another programmer reading/writing your program would be aware that this const variable is to not be changed.

- **Q: Why should I initialize a variable?**

  A: If you don't, you would not know for sure what the variable **contains** for a starting value.

  It would contain junk data at that memory location of the variable, and you cannot check against non-zero values of it cause you wouldn't know that it's not initialized with this check.

- **Q: Why shouldn't I use global variables frequently?**

  A: **Global variables** can be **read** and **assigned** globablly. The **latter** is the problem as they can be changed globally.

  Trivially, other programmers can mistakenly overwrite your global variables and that would effect your code—even in another .cpp file.

- **Q: What happens if I decrement from zero contained in an unsigned int**?

  A: You see a **wrapping effect**. Decrementing an **unsigned integer** that contains 0 by 1 means that it wraps to the **highest** value it can hold!

3.9 Quiz

- What is the difference btw a **signed** and an **unsigned integer**?

  A: A **signed** integer can store both positive and negative values, whereas an **unsigned** integer can store only non-negative values (0 and positive numbers). Since unsigned integers do not need to represent negative values, they can represent a larger maximum positive value.

- Why should you **not use** #define to declare constant?

  A: Because `#define` performs a simple text substitution and is not type-safe. Modern C++ prefers `const` or `constexpr` because they are checked by the compiler, have a type, and are easier to debug.

- Why would you initialize a variable?

  A: Initializing a variable gives it a known value before it is used, helping to prevent bugs caused by uninitialized (garbage) values.

- Consider the enum below. What is the value of Queen?

```
enum YourCards { Ace, Jack, Queen, King };
```

  A: `Queen` has the value `2`. By default, enumeration values begin at 0 and increase by 1.

- What is wrong with this variable name?

```
int Integer = 0;
```

  A: Nothing is technically wrong with the name. `Integer` is not a reserved C++ keyword, so it is a valid identifier. However, it is not a descriptive variable name and could be confused with the type `int`, so a more meaningful name is recommended.

4 Managing Arrays and Strings

4.1 What is an Array?

- An ***array*** is a group of elements forming a **complete** unit. Example: an array of solar panels.
  - is a collection of elements.
  - all elements contained in an array are of the **same kind**.
  - this collection forms a **complete set**.
- In C++, **arrays** enable you to **store** data elements of a type in **memory**, in a **sequential** and ordered fashion (cache friendly).

4.1.1 The Need for Arrays

- Imagine you want to store values for 500 integers.
- The best way to do this is with an array:

```
int arr[500];
```

- There are several common ways to initialize all integers in an array to `0`:

  1. Brace initialization (recommended)

     ```
     int arr[500] = {0};
     ```

  2. Empty brace initialization (C++11 and later)

     ```
     int arr[500] = {};
     ```

  3. Using `memset`

     ```
     int arr[500];
     memset(arr, 0, sizeof(arr));
     ```

  4. Using `std::fill`

     ```
     int arr[500];
     std::fill(arr, arr + 500, 0);
     ```

  5. Using a range-based `for` loop

     ```
     int arr[500];
     for (int &x : arr)
         x = 0;
     ```

  6. Using a traditional `for` loop

     ```
     int arr[500];
     for (int i = 0; i < 500; i++)
         arr[i] = 0;
     ```

  7. Partially initialize the first two elements

     ```
     int arr[5] = {10, 20};
     // arr = {10, 20, 0, 0, 0}
     ```

- **Note:** `memset` should only be used to initialize an integer array to `0`. It should *not* be used to initialize an array of integers to arbitrary values (e.g., `1`, `-1`, or `100`), since it operates on bytes rather than integer values.

  memset = sets raw memory bytes
  int initialization = sets typed values

  If you do `memset(arr, -1, sizeof(arr))` — this only "works as expected" in some cases because it produces all bits set:
  0xFF 0xFF 0xFF 0xFF = -1 (two's complement)

- Such **arrays** are called ***static arrays*** because the **number of elements** they contain as well as the **memory** the array consumes is **fixed** at **compile-time**.

4.1.2 How Data Is Stored in an Array

- Indices in C++ start with **0**.
- Memory of the elements in the array are equally allocated.
- The amount of **memory** reserved by the compiler for the array of 5 ints is:

```
sizeof(int) * 5
```

- In general, the amount of memory reserved by the compiler for an array in **bytes** is

```
Bytes consumed by an array = sizeof(element-type) * Number of Elements
```

4.1.3 Accessing Data Stored in an Array

- When you access an element at index N, the compiler does not "search" for it. Instead, it computes the exact memory address using **arithmetic**.
- Arrays in C/C++ are stored in **contiguous memory**, meaning each element is placed directly next to the previous one.
- The compiler treats the **array name** as a **pointer to the first element**. That is, the array name holds the memory address of index **0**.
- To access element $A[N]$, the compiler:
  - Starts from the base address $A[0]$
  - Adds an offset of $N \times$ `sizeof(element)`

  ```
  address of A[N] = base_address + N * sizeof(element)
  ```

- This is why arrays and pointers are closely related: `A` is essentially the address of `A[0]`, so:

  ```
  A[i] == *(A + i)
  ```

- In other words, pointer arithmetic is what makes array indexing possible.
- The C++ compiler does not check if the index is within the actual defined bounds of the array.
- Accessing an array beyond its bounds results in unpredicatble behavior and raises security risks to your program like crashing and should be avoided at all costs.
- **Listing 4.1** Arrays

4.1.4 Modifying Data Stored in an Array

- **Listing 4.2** Assigning Values to Elements in Array

4.2 Multidimensional Arrays

- A multidimensional array is an array of arrays to comprise in total an ***n-dimensional array***.
- The most common type is a **2D array**, which can be thought of as a table with rows and columns.
- Declaration example:

```
int grid[3][4];
```

  This creates a 2D array with 3 rows and 4 columns.

- Initialization example:

```
int grid[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

- Accessing elements uses two indices:

```
grid[row][col]
```

- Example:

```
grid[0][1] = 2;
grid[1][2] = 6;
```

- Memory layout: Multidimensional arrays are stored in **row-major order**, meaning rows are stored contiguously in memory.
- Example layout:

```
{1, 2, 3, 4, 5, 6}
```

- Pointer relationship:

```
grid[i][j] == *(*(grid + i) + j)
```

- Common use cases:
  - Matrices in math
  - Grids (games, maps)
  - Tables of data
- **Listing 4.3** Multidimensional Array

4.3 Dynamic Arrays

- A dynamic array is an array that can change size during runtime, unlike a normal C-style array which has a fixed size.
- In C++, the most common way to use dynamic arrays is through **vectors**.
- A vector is part of the Standard Template Library (STL) and automatically manages memory for you.
- Declaration example:

```
#include <vector>

vector<int> nums;
```

- Adding elements dynamically:

```
nums.push_back(10);
nums.push_back(20);
nums.push_back(30);
```

- Accessing elements (same as arrays):

```
cout << nums[0];   // 10
cout << nums[1];   // 20
```

- Size of the vector:

```
cout << nums.size();
```

- Example full usage:

```
vector<int> nums = {1, 2, 3};

for (int i = 0; i < nums.size(); i++)
{
    cout << nums[i] << " ";
}
```

- Key advantages:
  - Automatically resizes when elements are added
  - No need to manually manage memory
  - Safer than raw pointers and arrays
- Internally, vectors use dynamic memory allocation and may reallocate to a larger block when capacity is exceeded.

Side Notes:

Fixed Arrays using `std::array`

- `std::array` is a container from the C++ Standard Library that represents a fixed-size array.
- It is defined in the header:

```
#include <array>
```

- Unlike `std::vector`, the size of a `std::array` is fixed at compile time.
- Declaration example:

```
std::array<int, 5> nums = {1, 2, 3, 4, 5};
```

- Accessing elements works like normal arrays:

```
cout << nums[0];   // 1
cout << nums[3];   // 4
```

- Bounds-checked access can be done using `at()`:

```
cout << nums.at(0);
```

- Getting the size:

```
cout << nums.size();
```

- Iterating through a `std::array`:

```
for (int i = 0; i < nums.size(); i++)
{
    cout << nums[i] << " ";
}
```

- Range-based loop (modern C++):

```
for (int x : nums)
{
    cout << x << " ";
}
```

- Key advantages over raw arrays:
  - Knows its own size (`.size()`)
  - Safer access with `.at()`
  - Works well with STL algorithms
  - No pointer decay unless explicitly used
- Key difference from `std::vector`:
  - `std::array`: fixed size
  - `std::vector`: dynamic size

4.4 C-style Character Strings

- **C-style strings** are a special case of an array of characters.
- We have seen examples of C-style strings in the form of **string literals**:

```
std::cout << "Hellow World";
```

- This is equivalent to using the **array declaration**:

```
char sayHello[] = {'H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd', '\0'};
std::cout << sayHello << std::endl;
```

- Equivalent way using a string literal (recommended):

```
char sayHello[] = "Hello World";
std::cout << sayHello << std::endl;
```

- Equivalent using a pointer to a string literal:

```
const char* sayHello = "Hello World";
std::cout << sayHello << std::endl;
```

- Equivalent using std::string (modern C++ strings):

```
#include <string>

std::string sayHello = "Hello World";
std::cout << sayHello << std::endl;
```

- **Listing 4.6** Risky app of C-style strings
- If you forget the ***null terminator*** (`\0`) or it gets overwritten, then certain functions that rely on this **null-terminator** will go beyond the intended data and can cause data security issues and potentially a crash.
- All strings must end in `\0`.

- Example: `strlen()` computes the length of a string by walking the character buffer and counts the number of characters crossed until it reachs a **null-terminator** that indicates the **end of the string**.
- This behavior of `strlen()` makes it **dangerous** as it can easily walk past the bounds of the character array if this null-terminator isn't present.

- C programs use these kind of functions (`strlen()` (length of a string), `strcpy()` (copy a string), `strcat()` (concatenate a string), etc) that take in as input strings and can be dangerous as they seek the **null-terminator** and can exceed the boundaries potentially.

Side note:

4.4.1 Difference between '\n' and std::endl

- Both '\n' and std::endl are used to move output to a new line, but they are not the same.

  '\n' (newline character)

  - Inserts only a newline character into the output stream.
  - Does **not** flush the output buffer.
  - More efficient and faster.

```
cout << "Hello\n";
```

```
std::endl
```

  - Inserts a newline character.
  - Forces a **flush** of the output buffer.
  - Slower because flushing is an expensive operation.

```
cout << "Hello" << endl;
```

  Key difference

  - '\n' only moves to the next line.
  - std::endl moves to the next line **and flushes output immediately**.

  Comparison table

Feature	'\n'	std::endl
New line	Yes	Yes
Flush buffer	No	Yes
Performance	Faster	Slower

  When to use

  - Use '\n' for normal output (recommended most of the time).
  - Use std::endl when you need immediate output (e.g., debugging).

4.5 C++ Strings: std::string

- C++ standard strings are an efficient and **safer** way to deal with text input.
- `std::string` is **not static** in **size** like a **char array** implementation of a **C-style string** is.
- `std::string` can **dynamically resize** when more data is added, unlike fixed-size C-style character arrays. It is only slightly slower in theory due to overhead, but in practice and with compiler optimizations, it's minute (small difference in speed).
- **Listing 4.7** using std::string

4.6 QA

- **Q: Why take the trouble to initialize a static array's elements?**

  A: Unless initialized, the array, unlike a variable of any other tpye, contains **junk** and **unpredictable values** as the **memory** at that location was left untouched after the last operations.

  Initializing ensures a predictable initial state.

- **Q: Would you need to initialize the elements n a dynamic array for the same principles?**

  A: No, not necessarily. A **dynamic array** in cpp is smart. Elements are typically defaulted to a value of 0 if left uninitialized.

- **Q: Does the length of the string include the null-terminator**?

  A: No, it doesn't. The length of the string "Hello World" is 11, including the space character in btw the words and **excluding** the **null-terminator**.

- **Q: If i want to still use C-style arrays, what should be its size?**

  A: The size should be the length of the string + the null-terminator. So, basically account for a + 1 to the size when allocating the array.

  "Hello World" should be allocated in an array of size 11 + 1 = 12.

4.7 Quiz

1. If you need to allow the user to input strings, would you use C-style strings?

   A: No. As shown, the input can be greater than your bounds of the statically allocated C-style string.

   This will lead to undefined behavior if not checked.

2. How many characters are in `'\0'` as seen by the compiler?

   A: One character: the null-terminator.

5 Working with Expressions, Statements, and Operators

At its heart, a program is just a sequence of steps whether it be sequentially or in parallel.
These steps are programmed into expressions, statements, and use operators to perform calculations/actions.

5.1 Statements

- Languages — spoken or programmed — are made up of ***statements*** that are executed one after another.
- Example:

```
std::cout << "Hello World" << std::endl;
```

- All statements in C++ end in a **;**
- Extra spaces between statements **do not** matter (some languages it does matter like Python)
- In the front end of a compiler, the **lexical analyzer (lexer)** typically ignores whitespace that separates tokens. For example, the following declarations are treated identically:

```
char c;
char      c;
char
c;
char


c;
```

- The lexer produces the same sequence of tokens for each:

```
KEYWORD(char)  IDENTIFIER(c)  SEMICOLON(;)
```

- Whitespace is only significant in places where it separates tokens. Outside of string and character literals, extra spaces, tabs, and newlines generally do not change the meaning of a C++ program.
- Writing whitespaces within a statement can effect certain things like for example string literals:

```
cout << "Hello
 World" << endl; // is invalid
```

- To solve the above we can either do

```
cout << "Hello \
 World" << endl; // split ot two lines is OK
```

- Or this but entails a concatenation by the compiler

```
cout << "Hello "
 "World" << endl; // two string literals is also OK^{*}
     // (compiler does extra work --- catenation)
```

- The above one with \ is best to split up a long statement into multiple lines.

5.2 Compound Statements or Blocks

- When you group **statements** together within **braces** `{...}`, you create a ***compound statement*** or ***block***.

```
{
 int daysInYear = 365;
 cout << "Block contains an int and a cout statement" << endl;
}
```

- A ***block*** typically groups many statements to indicate that they belong together.
- We've seen it already through functions, constexpr's, arthimetic operations, and so on with conditionals, loops, etc

5.3 Using Operators

- ***Operators*** are tools for you to be able to work with data, transform it, process it, etc.

5.3.1 Assignment Operator (=)

- We've been using this

```
int daysInYear = 365;
```

- The **assignment operator** replaces the value contained by the ***operand*** on the left (called ***l-value***) by that on the right (called ***r-value***).

5.3.2 L-vales and R-values

- ***L-values*** often refer to **locations in memory**.
- A **variable** such as `daysInYear` from the preceding example is actually a **handle** to a **memory location** and is an ***l-value***.
- ***R-values***, on the other hand, can be the very **content** of a **memory location**.

- an l-value = a reference/handle to a memory location (something you can assign to, like a variable)
- an r-value = the value contained in that location (the data itself)

- So, all **l-values** can be used as **r-values**, but not all **r-values** can be used as **l-values**.

```
Example:
    daysInYear = 365;   // valid (l-value = r-value)

    365 = daysInYear;   // invalid (r-value cannot be assigned to)
```

- So, strictly speaking: an r-value **cannot** be an l-value.

5.3.3 Operators to Add (+), Subtract(-), Multiply(* or <<), Divide(/ or >>), and Modulo(%)

- These are basic arithmetic operators used to perform mathematical operations on numeric values.
- **Addition (+)**: Computes the sum of two operands.
- **Subtraction (-)**: Computes the difference between two operands.
- **Multiplication (* or <<)**: Multiplies two values; in some contexts, `<<` is used for bitwise left shift, which effectively multiplies by powers of 2.
- **Division (/ or >>)**: Divides one value by another; `>>` is a bitwise right shift, often corresponding to integer division by powers of 2.
- **Modulo (%)**: Returns the remainder after division of two integers.

- **Division (/)**: Performs division between two operands. If both operands are integers, the result is also an integer and the fractional part is truncated (i.e., decimal portion is discarded, effectively rounding toward zero). If at least one operand is a `float` or `double`, floating-point division is performed and the decimal part is preserved.

```
Key clarification (division behavior in C++)

7 / 2 = 3              (integer / integer → truncates toward zero)
7 / 2.0 = 3.5          (integer / double → floating-point division)
7.0 / 2 = 3.5          (double / integer → floating-point division)
7.0 / 2.0 = 3.5        (double / double → floating-point division)

-7 / 2 = -3            (truncates toward zero, NOT -4)
-7 / 2.0 = -3.5        (floating-point division)
-7.0 / 2 = -3.5        (floating-point division)

int x = 7 / 2;         // x = 3  (result truncated before assignment)
double y = 7 / 2;      // y = 3.0 (division happens first, still integer division, IMPORTANT)
double z = 7 / 2.0;    // z = 3.5 (floating-point division)
```

5.3.4 Operators to Increment(++) and Decrement(–)

- The increment operator (++) increases a variable's value by 1, and the decrement operator (–) decreases it by 1. These operators are shorthand for `x = x + 1` and `x = x - 1`.
- **Prefix form (++x, −−x)**: The variable is modified first, and the updated value is then used in the expression.
- **Postfix form (x++, x−−)**: The current value is used first, and then the variable is modified.
- Example:

```
int x = 5;
int a = ++x;  // x becomes 6, a = 6
int b = x++;  // b = 6, then x becomes 7
```

- **Pointers and increment/decrement**: When used on pointers, `++` and `--` move the pointer to the next or previous memory location of its type (not just +1 byte).
- Example:

```
int arr[] = {10, 20, 30};
int* p = arr;

p++;  // moves to arr[1]
p--;  // moves back to arr[0]
```

- **Iterators in loops**: In C++ iterators behave like pointers, so `++it` moves to the next element. This is commonly used in `for` loops to traverse containers.
- Example:

```
for (vector<int>::iterator it = v.begin(); it != v.end(); ++it)
{
 cout << *it << endl;
}
```

- **Prefix** is perferred over **postfix** due to, at least theoretically, with the **postfix operators**, the compiler needs to store the initial alue temporarily in the event of it needing to be assigned.

  The effect on performance in these cases is neglible with respect to integers, but in the case of certain classes there might be a point in this argument.

  Smart pointers may optimize away the differences.

5.3.5 Equality Operators (==) and (!=)

- (==) checks if two **operands** are equal.
- (!=) checks if two **operands** are not equal.
- The behavior of relational and equality operators can depend on the **types being compared**. For example, comparing integers vs floating-point numbers may involve implicit type conversion, and comparing pointers checks addresses rather than values.
- Example: integer vs floating-point comparison

```
int a = 5;
double b = 5.0;

cout << (a == b) << endl;  // true (a is promoted to double)
```

- Example: floating-point precision issues

```
double x = 0.1 + 0.2;
double y = 0.3;

cout << (x == y) << endl;  // false (precision error in floating-point math)


Why (0.1 + 0.2) != 0.3 in C++

Even though mathematically:
    0.1 + 0.2 = 0.3

In floating-point (binary) representation:
    0.1 is stored as an approximation
    0.2 is stored as an approximation
    0.3 is also stored as an approximation

So:
    (0.1 + 0.2) produces a slightly different value than 0.3

Example result:
    x = 0.1 + 0.2  ->  0.3000000000000000444
    y = 0.3        ->  0.2999999999999999889

Therefore:
    (x == y) is false due to tiny precision error
```

- Example: pointer comparison (compares addresses, not values)

```
int a = 10, b = 10;

int* p1 = &a;
int* p2 = &b;

cout << (p1 == p2) << endl;  // false (different memory locations)
cout << (a == b) << endl;    // true (values are equal)
```

- Example: string comparison (C-style strings compare addresses, not content)

```
const char* s1 = "hello";
const char* s2 = "hello";

cout << (s1 == s2) << endl;  // may be true or false (implementation-dependent)
```

5.3.6 Relational Operators

- Relational operators are used to compare two operands and determine the relationship between them. The result of any relational expression is a **boolean value** (true or false).
- **Less than** ($<$): Checks if the left operand is smaller than the right operand.
- **Greater than** ($>$): Checks if the left operand is larger than the right operand.
- **Less than or equal to** ($<=$): Checks if the left operand is smaller than or equal to the right operand.
- **Greater than or equal to** ($>=$): Checks if the left operand is larger than or equal to the right operand.
- Relational operators are commonly used in conditions such as `if` statements and loops.
- Example:

```
int a = 5, b = 10;

cout << (a < b) << endl;   // true
cout << (a > b) << endl;   // false
cout << (a <= 5) << endl;  // true
cout << (b >= 20) << endl; // false
```

- Relational comparisons can behave differently depending on types. For example, comparing integers is exact, while floating-point comparisons may be affected by precision errors as demostrated in the previous section.
- In C++, relational and logical expressions evaluate to a **boolean value** (`true` or `false`). When printed using `cout`, `false` is displayed as `0` and `true` is displayed as `1`.

  Any non-zero value is treated as `true` in conditional expressions.

  The compiler checks the condition against zero. So, if it's not zero, then the condition is marked true.

5.3.7 Logical Operations NOT, AND, OR, and XOR

- Logical operators are used to combine or modify boolean expressions. In C++, any expression evaluates to `true` (1) or `false` (0) when used in a logical context.
- **NOT (!) Operator**: Reverses the boolean value.

```
Operand: false
Result : !false = true (1)

Operand: true
Result : !true = false (0)
```

- **AND (&&) Operator**: Returns true only if both operands are true.

```
Operand 1: false   Operand 2: true
Result   : false && true = false (0)

Operand 1: true    Operand 2: true
Result   : true && true = true (1)
```

- **OR (||) Operator**: Returns true if at least one operand is true.

```
Operand 1: false   Operand 2: true
Result   : false || true = true (1)

Operand 1: false   Operand 2: false
Result   : false || false = false (0)
```

- **XOR concept (not a built-in logical operator in C++)**: Exclusive OR is true only when operands are different. In C++, it is implemented using bitwise XOR ^ for integers.

```
Operand 1: false (0)   Operand 2: true (1)
Result   : 0 ^ 1 = 1 (true)

Operand 1: true (1)    Operand 2: true (1)
Result   : 1 ^ 1 = 0 (false)
```

- **Key idea**: In C++, `false` is represented as 0 and `true` as 1 when printed, but internally they are boolean values, not integers.
- **Short-circuit behavior**:
  - `&&` stops evaluating if the first operand is false.
  - `||` stops evaluating if the first operand is true.

5.3.8 Bitwise Operators: NOT (~), AND (&), OR (|), XOR (^)

- Bitwise operators manipulate the individual **bits** of integral data types (such as `char`, `int`, and `long`). They operate on the binary representation of values rather than their decimal values.
- **Bitwise NOT (~)**: Flips every bit in the operand (0 becomes 1, and 1 becomes 0).
- Example:

```
unsigned char x = 10;      // 00001010
unsigned char y = ~x;      // 11110101
```

- **Bitwise AND (&)**: Produces a 1 in each bit position only if the corresponding bits of both operands are 1.
- Example:

```
12 & 10

1100
1010
----
1000    // 8
```

- **Bitwise OR (|)**: Produces a 1 in each bit position if either corresponding bit is 1.
- Example:

```
12 | 10

1100
1010
----
1110    // 14
```

- **Bitwise XOR (\^)**: Produces a 1 in each bit position only when the corresponding bits are different.
- Example:

```
12 ^ 10

1100
1010
----
0110    // 6
```

- Bitwise operators are commonly used for bit masking, setting or clearing flags, toggling bits, low-level hardware programming, and performance-sensitive code.

5.3.9 Bitwise right shift >> and left shift <<

- Bitwise shift operators move the bits of an integral value to the left or right by a specified number of positions.
- **Left Shift (<<)**: Shifts all bits to the left. Each left shift by one position is approximately equivalent to multiplying the value by 2, provided no significant bits are discarded (overflow).
- Example:

```
5 << 1

00000101   (5)
<< 1
--------
00001010   (10)

5 << 2

00000101   (5)
<< 2
--------
00010100   (20)
```

- **Right Shift (>>)**: Shifts all bits to the right. Each right shift by one position is approximately equivalent to integer division by 2, discarding any remainder.
- Example:

```
20 >> 1

00010100   (20)
>> 1
--------
00001010   (10)

20 >> 2

00010100   (20)
>> 2
--------
00000101   (5)
```

- Bits shifted out of the value are discarded. For a left shift, zeros are shifted into the least significant bits. For unsigned integers, a right shift shifts zeros into the most significant bits.
- Example:

```
1 << 5

00000001
<< 5
--------
00100000   (32)
```

- Bitwise shift operators are commonly used for efficient multiplication or division by powers of two, bit masking, manipulating binary data, and low-level systems programming.

- **Note**: Although left and right shifts can often be thought of as multiplication or division by powers of two, this relationship is only reliable when no overflow occurs (for left shifts) and when shifting unsigned integers.

5.3.10 Compound Assignment Operators

- Compound assignment operators combine an arithmetic or bitwise operation with assignment. They provide a shorthand for updating the value of a variable.
- **Addition Assignment (`+=`)**: Adds the right operand to the left operand, then stores the result in the left operand.

```
x += 5;    // Equivalent to: x = x + 5;
```

- **Subtraction Assignment (`-=`)**: Subtracts the right operand from the left operand, then stores the result.

```
x -= 5;    // Equivalent to: x = x - 5;
```

- **Multiplication Assignment (`*=`)**: Multiplies the left operand by the right operand, then stores the result.

```
x *= 5;    // Equivalent to: x = x * 5;
```

- **Division Assignment (`/=`)**: Divides the left operand by the right operand, then stores the result.

```
x /= 5;    // Equivalent to: x = x / 5;
```

- **Modulo Assignment (`%=`)**: Computes the remainder after division and stores the result.

```
x %= 5;    // Equivalent to: x = x % 5;
```

- **Bitwise AND Assignment (`&=`)**: Performs a bitwise AND with the right operand and stores the result.

```
x &= y;    // Equivalent to: x = x & y;
```

- **Bitwise OR Assignment (`|=`)**: Performs a bitwise OR with the right operand and stores the result.

```
x |= y;    // Equivalent to: x = x | y;
```

- **Bitwise XOR Assignment (`^=`)**: Performs a bitwise XOR with the right operand and stores the result.

```
x ^= y;    // Equivalent to: x = x ^ y;
```

- **Left Shift Assignment (`<<=`)**: Shifts the bits of the left operand to the left and stores the result.

```
x <<= 2;   // Equivalent to: x = x << 2;
```

- **Right Shift Assignment (`>>=`)**: Shifts the bits of the left operand to the right and stores the result.

```
x >>= 2;   // Equivalent to: x = x >> 2;
```

- Compound assignment operators are commonly used to write shorter, more readable code while avoiding repetition of the left-hand operand.

5.3.11 Using sizeof Operator

- `sizeof` operator tells you the **amount of memory** in **bytes** consumed by a particular type or a variable.

```
sizeof(variable);
  or
sizeof(type);
```

- `sizeof()` may look like a function, but it is **not a function**; it's an **operator**.
- Interestingly, this operator **cannot be overloaded**, i.e. **cannot** be defined by the programmer.

5.3.12 Operator Precedence

1. Scope resolution
2. Member selection, subscripting, increment, decrement
3. sizeof, prefix increment and decrement, complement, and, not, unary minus and plus, address-of and dereference, new, new[], delete, delete[], ccasting, sizeof()
4. Member selection for pointer
5. Multiply, divide, modulo
6. Add, subtract
7. Shift (shift left, shift right)
8. Inequality relational
9. Equality, inequality
10. Bitwise AND
11. Bitwise exclusive OR
12. Bitwise OR
13. Logical AND
14. Logical OR
15. Conditional
16. Assignment operators
17. Comma

5.4 QA

- Trivial questions

5.5 Quiz

1. I am writing an app to divide numbers. What's a better suited data type: int or float?
   A: I would say float personally, cause it wouldn't truncate the decimals.

6 Controlling Program Flow

- **Switch statements** instead of if conditions
- The `switch (variable)` must match the `case variable_type:`
- If you dont do a `break;` in the cases, you **fall through** to the next below case.

```
#include <iostream>

using namespace std;

int main() {
    int choice;

    cout << "Enter a number (1-3): ";
    cin >> choice;

    switch (choice) {
        case 1:
            cout << "You selected option 1" << endl;
            break;

        case 2:
            cout << "You selected option 2" << endl;
            break;

        case 3:
            cout << "You selected option 3" << endl;
            break;

        default:
            cout << "Invalid option" << endl;
            break;
    }

    return 0;
}
```

7 Organizing Code with Functions

8 Pointers and References

8.1 What is a Pointer?

- A ***pointer*** is also a **variable** — one that **stores** an address in **memory**.
- Just the same way as a **variable** of type **int** is used to contain an **integer value**, a **pointer variable** is used to contain a **memory address**.
- Thus, a **pointer** is a **variable**, and like all variables a **pointer** occupies **space in memory**.
- The **value** contained in a **pointer** is interpreted as a **memory address**.
- Again, a **pointer** is a special variable that *points* to a **location in memory**.

- Note: **memory addresses** are in ***hexadecimal (base 16)*** and start with an `0x`

8.1.1 Declaring a Pointer

- Just like a variable, a pointer needs to be declared.
- You normally declare a pointer to point to a specific **type** (ex: int*)
- This would mean that the **address** contained in the **pointer** points to a **location in the memory** that holds an integer.
- A ***void pointer*** (`void*`) is a pointer that can store the address of any data type. It is often used to represent a generic pointer to raw memory. However, since it has no type information, it cannot be dereferenced directly and must first be cast to a specific pointer type.

```
#include <iostream>
using namespace std;

int main()
{
    int x = 10;
    double y = 3.14;

    void* ptr;

    ptr = &x;   // void pointer holds address of int
    cout << *(static_cast<int*>(ptr)) << endl;

    ptr = &y;   // now holds address of double
    cout << *(static_cast<double*>(ptr)) << endl;

    return 0;
}
```

- As in the case with most variables, unless you initialize a pointer it will contain a random value.
- To avoid accessing a **random (uninitialized) memory address**, pointers should be initialized to **NULL** (C style) or **nullptr** (C++11 and later). This ensures the pointer is explicitly set to point to nothing until it is assigned a valid address.
- **nullptr** is preferred in modern C++ because it is type-safe, whereas **NULL** is typically just defined as 0 and may cause ambiguity in overloaded function calls.
- Thus, declaring a pointer to an integer would be

```
int *pointsToInt = NULL; // NULL lol
```

8.1.2 Determing the Address of a Variable Using the Reference Operator (&)

- if `varName` is a variable, `&varName` gives the **address in memory** where its value is placed.
- So, if you have declared an integer `int age = 30;`, `&age` would be the **address in memory** where the value 30 is placed.
- **Listing 8.1** Addresses VERY IMPORTANT

8.1.3 Access Pointer Data Using the Dereference Operator (*)

- The dereference operator (`*`) is used to access the value stored at the memory address held by a pointer.
- Instead of working with the address itself, dereferencing allows you to work with the **actual data stored at that address**.
- Example:

```
int x = 10;
int *p = &x;

cout << p << endl;   // prints address of x
cout << *p << endl;  // prints value of x (10)
```

- The expression `*p` means "go to the memory location stored in p and access its value."
- If the value at that address is changed through the pointer, the original variable is also modified.
- Example:

```
int x = 10;
int *p = &x;

*p = 20;   // modifies x through the pointer

cout << x << endl;   // outputs 20
```

- Dereferencing only works on valid (non-null) pointers. Using `*` on an uninitialized or null pointer leads to undefined behavior.

- For example, just look at **Listing 8.1**

- When the ***dereference operator (*)*** is used, the program uses the **address stored in the pointer** to access the **object located at that memory address**. The amount of memory accessed depends on the **data type of the pointer** (e.g., `int`, `double`), as determined by `sizeof(type)` on the given system.
- The **dereference operator** is also called the ***indirection operator*** (never heard that in my life)
- Basically, the **memory address** stored in a **pointer** is very important and can cause issues if the address is invalid.

8.1.4 What is sizeof() a Pointer?

- The **pointer** is just another **variable** that contains a **memory address**.
- Hence, **irrespsecitve** of the **type** that is being pointed to, the content of a **pointer** is an **address** — a number.
- The **length of an address**, that is the **number of bytes** required to store it, is a **constant** for a given system.
- The `sizeof()` a **pointer** is hence **dependent** on the **compiler** and the **operating system** the program has been compiled for and is **not dependent** on the nature of the **data** being pointed to.
- **Listing 8.6**
- The output of the `sizeof(int*)` would be
  - **4 bytes** on a **32-bit compiler** and
  - **8 bytes** on a **64-bit compiler**.

8.2 Dynamic Memory Allocation

Dynamic memory allocation allows a program to **request memory at runtime** from the **heap** instead of the **stack**.
This is useful when the **required memory size** is **not known** at **compile time**.

The `sizeof()` operator is very useful here.

8.2.1 Using new and delete or delete[]

- In C++, dynamic memory is allocated using the **new** operator and released using **delete**.
- For arrays, **new**[] and **delete**[] must be used.
- Example:

```
int *p = new int;   // allocate single integer
*p = 10;

cout << *p << endl;

delete p;           // free memory
```

- Example (arrays):

```
int *arr = new int[5];

for (int i = 0; i < 5; i++)
{
    arr[i] = i * 10;
}

for (int i = 0; i < 5; i++)
{
    cout << arr[i] << " ";
}

delete[] arr;   // free array memory
```

- **Important:** Using **delete** on memory allocated with **new**[] (or vice versa) leads to undefined behavior.

8.2.2 Using malloc, realloc, etc and free

- In C (also available in C++), dynamic memory is managed using **malloc**, **calloc**, **realloc**, and **free** from the `<cstdlib>` library.
- **malloc**: Allocates a block of memory (uninitialized).

```
int *p = (int*) malloc(5 * sizeof(int));

for (int i = 0; i < 5; i++)
{
    p[i] = i + 1;
}

free(p);
```

- **calloc**: Allocates memory and initializes it to zero.

```
int *p = (int*) calloc(5, sizeof(int));
```

- **realloc**: Resizes previously allocated memory block.

```
p = (int*) realloc(p, 10 * sizeof(int));
```

- **free**: Releases memory allocated by malloc/calloc/realloc.

```
free(p);
```

- **Important difference**: Unlike **new/delete**, these functions do not call constructors or destructors in C++ and should be used carefully in modern C++ code.

Important side note:

- In C-style dynamic memory allocation, functions like `malloc`, `calloc`, and `realloc` return a ***void pointer (void*)***, which is a generic pointer type that can hold the address of any data type.
- A `void*` is used because these functions do not know in advance what type of data the allocated memory will store. Instead, they simply return a raw memory address from the heap.
- Example:

```
int *p = (int*) malloc(5 * sizeof(int));
```

- The cast `(int*)` is required in C++ because a `void*` cannot be automatically converted into a typed pointer without an explicit conversion.
- **Why malloc returns void*:**
  - It allows a single allocation function to be **generic** and work with any data type.
  - It represents raw, untyped memory returned from the heap.
  - The programmer decides how to interpret that memory by casting it to the appropriate pointer type.
- Example showing interpretation:

```
void* ptr = malloc(4 * sizeof(int));

int* iptr = (int*) ptr;
iptr[0] = 10;
```

- **Important:** Unlike **new**, these functions do not perform type construction or initialization—they only allocate raw memory.

- In C++, it is recommended to use `static_cast` instead of C-style casts when converting a `void*` to a typed pointer.
- Example:

```
void* ptr = malloc(4 * sizeof(int));

int* iptr = static_cast<int*>(ptr);

// or really just

int* cppiptr = new int;

iptr[0] = 10;
```

- The expression `(int*) ptr` is a C-style cast, while `static_cast<int*>(ptr)` is the safer and more explicit C++-style cast.
- **Note:** The cast is required in C++ because a `void*` cannot be implicitly converted to another pointer type.

Additional important side notes:

- The **new operator** in C++ can also be **overloaded** for custom memory management in user-defined classes. This allows a class to define how memory is allocated when `new` is used.
- Example:

```
class A {
public:
    void* operator new(size_t size);
};

void* A::operator new(size_t size)
{
    cout << "Custom new called" << endl;
    return malloc(size);
}
```

- In this example, the class `A` defines its own version of the `new` operator. The parameter `size_t size` represents the number of bytes required to allocate an object of type `A`.
- **Note:** Although the operator returns `void*` internally in the declaration above, in practice it is typically expected to return a pointer of type `A*` after allocation.

- **Listing 8.7** new and delete with operator *
- **Operator *delete*** canot be invoked on any address containing a pointer, rather only those that have been returned by the **operator *new*** and **only** those that have not already been released by a *delete*.
- **Listing 8.8**

Side note:

8.2.3 Operators vs Functions vs constexpr

- In C++, computation can be expressed using **operators**, **functions**, and **constexpr expressions**. Each serves a different purpose in how code is written and evaluated.
- **Operators**: Special symbols that perform built-in operations on operands. They are concise and have fixed precedence and associativity rules.

```
int x = 5 + 3;   // + is an operator
```

- **Functions**: Named blocks of reusable code that perform a task. They are called explicitly and can contain complex logic.

```
int add(int a, int b)
{
    return a + b;
}

int x = add(5, 3);
```

- **constexpr**: A specifier that tells the compiler that a value or function can be evaluated at **compile time** if given constant inputs. This improves performance and enables compile-time computation.
- Example of constexpr variable:

```
constexpr int x = 5 + 3;   // evaluated at compile time
```

- Example of constexpr function:

```
constexpr int square(int x)
{
    return x * x;
}

int y = square(4);   // may be computed at compile time
```

- **Key differences**:
  - Operators: built-in symbolic operations with fixed rules
  - Functions: runtime or compile-time logic (depending on usage)
  - constexpr: guarantees compile-time evaluation when possible
- **Summary**:
  - Operators → concise symbolic computation
  - Functions → reusable procedural logic
  - constexpr → compile-time computable values/functions

- Operators and functions are handled differently by the compiler. Function calls use an explicit argument list in parentheses, while operators are part of the C++ language grammar and are parsed as expressions rather than function calls.

- For example, a function call is parsed as:

  ```
  add(5, 3);
  ```

- In contrast, an operator expression is parsed using syntax rules defined by the language:

  ```
  5 + 3
  ```

- This difference comes from the compiler's parser: function calls follow a "function-call grammar rule", while operators follow "expression grammar rules" with precedence and associativity.
- This is why operators such as `+`, `-`, and `new` do not use function-style parentheses for arguments, even though they conceptually operate on operands.
- In summary, functions are explicitly invoked using argument lists, while operators are built into the language syntax and are interpreted as part of expressions during parsing.

- When the compiler parses an expression, it first performs lexical and syntactic analysis. If it encounters an **identifier**, it determines whether it refers to a variable, function, or other named entity. If it encounters an **operator**, it applies the language grammar rules for expressions, using operator precedence and associativity to determine how operands are grouped and evaluated.
- For example, in a function call like:

  ```
  add(5, 3);
  ```

  the compiler treats `add` as an identifier referring to a function and processes the parentheses as a function-call argument list.
- In contrast, in an expression like:

  ```
  5 + 3 * 2
  ```

  the compiler recognizes `+` and `*` as operators and constructs an expression tree based on precedence rules, evaluating multiplication before addition.

- When the compiler encounters a custom operator, it does not introduce new syntax rules. The expression is first parsed using the same grammar rules as built-in operators. For example, an expression like `a + b` is always parsed as a binary operator expression.
- During semantic analysis, the compiler then performs **operator overload resolution**. It checks whether a valid function named `operator+` exists for the given operand types, either as a member function or a non-member function.
- For example, the expression:

  ```
  a + b
  ```

  may be resolved internally as:

  ```
  a.operator+(b)
  ```

- If a matching overload is found, the compiler replaces the operator expression with the corresponding function call. If no valid overload exists, a compilation error occurs.

- In summary, custom operators are not new language constructs; they are existing operators that are bound to user-defined functions during the overload resolution phase of compilation.

- In C++, operators have a fixed number of operands (called **arity**) determined by the language grammar. Operators cannot accept an arbitrary number of arguments like functions.
- **Unary operators** operate on one operand:

```
!a
++a
-a
```

- **Binary operators** operate on two operands:

```
a + b
a < b
a == b
```

- **Ternary operator** is the only operator with three operands:

```
condition ? expr1 : expr2
```

- The compiler does not allow changing the arity of an operator. For example, `a + b + c` is not a three-operand addition operator, but is instead parsed as:

```
(a + b) + c
```

- During compilation, the expression is first parsed using fixed grammar rules (unary, binary, or ternary forms), and then overload resolution determines whether a matching `operator+`, `operator!`, etc., function exists for the given operand types.
- In summary, operator argument count is fixed by the language, and the compiler handles operators by first parsing expression structure and then binding them to appropriate overloaded functions if available.

8.2.4 Operator Overloading Examples (Unary, Binary, and Ternary-like)

- **Unary operator example (prefix ++):**

```
#include <iostream>
using namespace std;

class A {
    int x;

public:
    A(int val) : x(val) {}

    // Unary prefix ++
    A operator++()
    {
        ++x;
        return *this;
    }

    void show()
    {
        cout << x << endl;
    }
};

int main()
{
    A obj(5);
    ++obj;
    obj.show();   // 6
}
```

- **Binary operator example (+):**

```
#include <iostream>
using namespace std;

class A {
    int x;

public:
    A(int val) : x(val) {}

    // Binary +
    A operator+(const A& other)
    {
        return A(x + other.x);
    }

    void show()
    {
        cout << x << endl;
    }
};

int main()
{
    A a(5), b(10);
```

```
    A c = a + b;

    c.show();   // 15
}
```

- **Ternary operator (?:) note**:
  - The ternary operator `?:` cannot be overloaded in C++.
  - It is built into the language grammar and cannot be redefined.
- **Ternary-like behavior using a function**:

```
#include <iostream>
using namespace std;

class A {
    int x;

public:
    A(int val) : x(val) {}

    void show()
    {
        cout << x << endl;
    }
};

A choose(bool cond, A a, A b)
{
    return cond ? a : b;
}

int main()
{
    A a(5), b(10);

    A c = choose(true, a, b);
    c.show();   // 5
}
```

- In an expression like `a + b`, the operator `+` is not a standalone operation. If `operator+` is defined as a member function inside class `A`, the expression is interpreted as a function call on the left-hand object.
- For example:

```
A c = a + b;
```

- The compiler rewrites this as:

```
A c = a.operator+(b);
```

- This means the operator "belongs" to the left operand (`a`), and the right operand (`b`) is passed as an argument.
- In general:
  - `a + b` $\longrightarrow$ `a.operator+(b)`

  - `b + a` $\longrightarrow$ `b.operator+(a)`

- This is why operator order matters when using member operator overloading. If symmetry is required, a non-member (friend) operator overload is often used instead.

- **To create an overloaded operator function, you write *operator* followed by the operator symbol.**

8.2.5 Incrementing (++) and Decrementing (−− on Pointers)

- A **pointer** contains a **memory address**, ex: `0x002EFB34` — the **address** where, say, an integer is placed.
- The **integer** is **4 bytes** long on our system, so it **occupies 4 places in memory** from:
  - `0x002EFB34` to `0x002EFB37` inclusive
- **Incrementing** the **pointer** using the **operator** (++) would **not** result in the **pointer** pointing to `0x002EFB35` (notice the 4 to the 5), for pointing to the middle of the integer would be pointless.

- An **increment** or **decrement** operation on a **pointer** is interpreted by the **compiler** as your need to point to the **next value** in the **block of memory**, assuming it would be of the same **type**, and **not** to the **next byte** (unless the value type is **1 byte** large, like a `char` for instance).

  Example:

  Suppose we have the following code:

  ```
  int arr[] = {10, 20, 30, 40};
  int *p = arr;
  ```

  Assume an `int` occupies **4 bytes** in memory. The array could be stored as follows:

Address	Value
1000	10
1004	20
1008	30
1012	40

  Initially, `p` points to address `1000`, so:

  ```
  *p    // 10
  ```

  After incrementing the pointer:

  ```
  p++;
  ```

  the compiler knows that `p` is an `int *`, so it advances the pointer by `sizeof(int)` (4 bytes), not by 1 byte. The pointer now points to address `1004`:

Address	Value
1000	10
1004 ← p	**20**
1008	30
1012	40

  Thus,

  ```
  *p    // 20
  ```

Incrementing the pointer again moves it to address `1008`, where:

```
*p    // 30
```

In general, for a pointer of type `T *`, the operation `p++` advances the pointer by `sizeof(T)` bytes, making it point to the **next object** of type `T` in memory rather than simply the next byte.

- So, **incrementing** a **pointer** such as

  ```
  int age = 30;
  int* pointsToInt = &age;
  ```

  results it being **incremented** by **4 bytes**, which is the `sizeof(int)`.

- **It increments based on the size of the type it points to, not the size of the pointer itself.**

- Using ++ on this **pointer** is telling the compiler that you want it to point to the **next consecutive integer**.

  Hence, after incrementing, the pointer would then point to `0x002EFB38` (from 4 to 8 in hex)

- Similarly, adding 2 to this pointer would result in it moving 2 **integers** ahead, that is **8 bytes** (in this case).

- **Decrementing** pointers using −− gives the same effect but you reduce by the sizeof what the pointer is pointing to.

8.2.6 Using the const keyword on Pointers

- Lesson 3 we learned that declaring a **variable** as a `const` ensures that value of the variable is **fixed** as the initialization value for the **life** of the variable.
- The value of a **const-variable** cannot be changed, and therefore it cannot be used as an **l-value**.

- **Pointers** are **variable** too, and hence the `const` keyword can be used here.
- **However**, **pointers** are a **special kind of variable** as they contain a **memory address** and are used to modify **memory** at that address.
- Thus, when it comes to **pointers** and **constants**, you have the following combinations:

1. The **address** contained in the **pointer** is **constant** and **cannot be changed**, yet the **data** at that address **can be changed**.

```
int daysInMonth = 30;
int* const pDaysInMonth = &daysInMonth;
*pDaysInMonth = 31; // OK! Data pointed to can be changed
int daysInLunarMonth = 28;
pDaysInMonth = &daysInLunarMonth; // NOT OK! Cannot change address!
```

Note here also, the **const** keyword is after the int*!!!

```
const int *p;
^^^^^^^^^
data is const (the int being pointed to is const)

int *const p;
     ^^^^^
pointer is const (the pointer variable p cannot be changed)
```

Also

```
Valid:

const int x = 5;
int const x = 5;

Invalid:

int x const = 5;   // not allowed in C/C++

-----

const int x -> x is a const int
int const x -> same thing (just different order)
int x const -> not legal grammar in C/C++
```

2. **Data** pointed to is **constant** and **cannot be changed**, yet the **address** contained in the pointer **can be changed** — that is, the **pointer** can also point elsewhere:

```
int hoursInDay = 24;
const int* pointsToInt = &hoursInDay; // OK! address can be changed but not the data.
int monthsInYear = 12;
pointsToInt = &monthsInYear; // OK again for same reason
*pointsToInt = 13; // NOT OK! Cannot change data being pointed to

int* newPointer = pointsToInt; // NOT OK! Cannot assign const to non-const
// cause this would entail you being able to change data.
// const int *p  means:
// "I promise I will not modify the int through this pointer"
// int* newPointer = pointsToInt; violates that contract.
```

3. Both the **address** conatined in the pointer and the **value** being pointed to are **constant** and cannot be changed (most restrictive variant):

```
int hoursInDay = 24;
const int* const pHoursInDay = &hoursInDay;

*pHoursInDay = 25; // NOT OK! Cannot change data being pointed to.

int daysInMonth = 30;
pHoursInDay = &daysInMonth; // NOT OK! Cannot change address.
```

- These different forms of **const** is particularly useful when passing **pointers** to **functions**.
- Function parameters need to be declared to support the highest possible (restrictive) level of **constness**, to ensure that the function does **not** modify the pointed value when it is not supposed to.

8.2.7 Passing by ref C vs C++

```
// C++ only (reference)
void foo(int& v) {
    v = 5;
}

int main() {
    int v = 0;
    foo(v);
}

// C/C++ compatible (pointer to int)
void foo(int* v) {
    *v = 5;
}

int main() {
    int v = 0;
    foo(&v);
}
```

8.2.8 Clarification for pointer declarations

The confusing part is that the `*` is **part of the declaration**, not part of the type that is being made `const`.

Step 1: A Regular Variable

A normal integer declaration is:

```
int x;
```

If we make the variable constant:

```
const int x;
// or equivalently
int const x;
```

the `int` itself becomes **constant**.

Step 2: Introduce a Pointer

Consider the declaration:

```
int *p;
```

This is read as:

> "`p` is a pointer to an `int`."

Notice that there are now **two different things (or both to be the most restrictive!)** that could be made constant:

1. the **integer** being pointed to,
2. the **pointer** itself.

Case 1: Make the Integer Constant

Starting from:

```
int *p;
```

we can make the integer constant:

```
const int *p;
```

The `const` applies to the `int`, not to the pointer.
You can think of it conceptually as:

```
(const int) *p
```

This means:

- the integer cannot be modified through `p`,
- the pointer itself may still point somewhere else.

Conceptually:

```
p
|
v
+-----+
| 42  |   <- const int
+-----+
```

Therefore,

```
*p = 10;        // NOT OK
p = &otherInt;  // OK
```

Case 2: Make the Pointer Constant

Again start with:

```
int *p;
```

This time make the pointer constant:

```
int *const p;
```

Here, the `const` appears *after* the `*`, so it applies to the pointer itself.
Conceptually:

```
(int *) const p
```

This means:

- the pointer cannot be changed,
- the integer being pointed to may still be modified.

Conceptually:

```
const p
|
v
+-----+
| 42  |
+-----+
```

Therefore,

```
*p = 10;        // OK
p = &otherInt;  // NOT OK
```

Why Doesn't `const int *` Mean "Constant Pointer"?

This is because, in C/C++, the `*` belongs to the **declarator** (the variable being declared), not to the base type.
For example,

```
int *p, q;
```

does **not** declare two pointers.
Instead, it declares:

```
int *p;   // pointer
int q;    // ordinary int
```

If the `*` were part of the type, then both `p` and `q` would be pointers, but they are not.
Therefore, the compiler interprets

```
const int *p;
```

as

```
(const int) *p
```

rather than

```
const (int *)
```

Summary

There are two separate objects involved in a pointer declaration:

1. the object being pointed to,
2. the pointer itself.

The position of `const` determines which one is constant:

Declaration	Meaning
`int *p`	pointer to an int
`const int *p`	pointer to a constant int
`int *const p`	constant pointer to an int
`const int *const p`	constant pointer to a constant int

A useful way to read declarations is:

- `const int *p` → "`p` is a pointer to a **constant int**."
- `int *const p` → "`p` is a **constant pointer** to an int."

Once you recognize that there are **two different things** that can be made constant (the pointee and the pointer), the placement of `const` becomes much more intuitive.

- **Add const (always allowed):**

```
int *p
const int *p      (OK: add const to data)
int *const p      (OK: add const to pointer)
const int *const p (OK: add const to both)
```

- **Remove const (NOT allowed implicitly):**

```
const int *p  -> int *p      (NOT OK)
int *const p  -> int *p      (NOT OK)
const int x   -> int x       (NOT OK)
```

- **Safe conversions (const-preserving):**

```
int *p        -> const int *p   (OK)
int x         -> const int x    (OK)
int *p        -> int *const p   (OK only when initializing)
```

- **Pointer rule intuition:**

```
You can always promise "I won't modify",
but you cannot take back that promise implicitly.
```

8.2.9 Passing Pointers to Functions

- Pointers are an effective way to **pass memory space** that contains relevant data for functions to work on.
- The **memory space** shared can also return the result of an **operation**.
- Given the previous section, you should modify pointers as parameters to be const or not if you want to restrict the callee modifiying the pointer or the value pointed to or both.
- **Listing 8.10** Use const in pointer function parameters

8.2.10 Pointers in Static vs Dynamic Memory

- **Pointers are not tied only to dynamic memory.** Pointers can store the address of variables in **stack (static)** memory or **heap (dynamic)** memory.
- For example, static arrays are just a single pointer underneath and we go to each element via adding the sizeof(element_type) to the current pointer and increment or decrement the address by that size in bytes (cause `sizeof` returns bytes)
- **Static (stack) memory example:**

```
int x = 10;
int *p = &x;
```

  - `x` is stored in stack memory
  - `p` simply stores its address
  - no dynamic allocation is involved

- **Array example (contiguous static memory):**

```
int arr[] = {1, 2, 3};
int *p = arr;
```

  - arrays decay into pointers in expressions
  - pointer arithmetic allows traversal of elements
  - `p++` moves to the next array element, not next byte

- **Dynamic (heap) memory example:**

```
int *p = malloc(sizeof(int));
*p = 42;

// or

int *p = new int;
*p = 42;
```

  - memory is allocated on the heap
  - pointer is required to access and manage that memory
  - must be manually freed in C using `free(p)` or `delete ptr` or `delete[] ptr` in C++

- **Function parameter example (pass by address):**

```
void foo(int *p) {
    *p = 100;
}

int main() {
    int x = 5;
    foo(&x);
}
```

  - allows modification of original variable
  - avoids copying large data structures

- **Key insight:** Pointers are a **general mechanism for storing memory addresses**, not a feature exclusive to dynamic memory.
- **Exam takeaway:**

– stack variables can have pointers
– heap variables require pointers
– arrays behave like pointers in most expressions
– pointer arithmetic works based on data type size, not byte-by-byte movement

8.3 Common Programming Mistakes when using Pointers

- **Uninitialized pointers (wild pointers)**

```
int *p;
*p = 10;   // undefined behavior
```

  - Pointer is not initialized to valid memory.
  - Must always assign a valid address before dereferencing.

- **Dangling pointers (use-after-free)**

```
int *p = malloc(sizeof(int));
free(p);
*p = 5;   // undefined behavior
```

  - Memory has been freed but pointer still holds the old address.
  - Accessing it leads to undefined behavior.

- **Memory leaks**

```
int *p = malloc(sizeof(int));
p = NULL;   // lost reference to allocated memory
```

  - Original allocated memory is no longer reachable.
  - This causes memory that cannot be freed (leak).

- **Null pointer dereference**

```
int *p = NULL;
*p = 10;   // undefined behavior
```

  - NULL does not point to valid memory.
  - Dereferencing NULL leads to crashes or undefined behavior.

- **Incorrect pointer types**

```
int x = 10;
float *p = &x;   // type mismatch
```

  - Pointer type must match the type of the object.
  - Otherwise memory is interpreted incorrectly.

- **Confusing pointer assignment vs dereference**

```
int x = 5;
int *p = &x;

p = 10;   // incorrect: changes pointer, not value
*p = 10;  // correct: changes value at address
```

  - `p` stores an address.
  - `*p` accesses the value at that address.

- **Out-of-bounds pointer access**

```
int arr[3] = {1, 2, 3};
int *p = arr;

p[5] = 10;   // undefined behavior
```

  - Pointer arithmetic must stay within allocated memory.
  - Accessing outside bounds is undefined behavior.

8.4 References

- A ***reference*** is an **alias** for an existing variable.
- Once a reference is created, it **must be initialized immediately**.
- A reference does not store a separate copy of the data; it refers to the **same memory location** as the original variable.
- Any operation on the reference is performed on the **original variable**.

- References are declared using the **reference operator (&)**:

```
VarType original = value;
VarType& ref = original;
```

- After binding, the reference becomes an alternative name for the same variable:

```
ref = 10;   // changes original as well
```

- **Important properties of references:**
  - Must be initialized at declaration
  - Cannot be null (unlike pointers)
  - Cannot be reseated to refer to another variable
  - Acts as an automatic dereference (no need for *)

- **Example:**

```
int x = 5;
int& r = x;

r = 10;   // x is now 10
```

- **Key intuition:** A reference behaves like the original variable, just under a different name.

8.4.1 References as Function Parameters

- References are commonly used as **function parameters** in C++ to allow functions to modify the original argument.
- When a parameter is declared as a reference, the function receives an **alias** to the original variable, not a copy.

- **Pass-by-value vs Pass-by-reference:**

```
// Pass-by-value (copy)
void foo(int x) {
    x = 10;   // does NOT affect original
}

// Pass-by-reference (no copy)
void bar(int& x) {
    x = 10;   // modifies original
}
```

- In pass-by-value:
  - A copy of the argument is created
  - Changes inside the function do not affect the original variable
- In pass-by-reference:
  - No copy is made
  - The parameter becomes an alias for the original variable
  - Changes inside the function affect the caller's variable

- **Example:**

```
void increment(int& x) {
    x++;
}

int main() {
    int a = 5;
    increment(a);
    // a is now 6
}
```

- **Why use references in parameters?**
  - Avoid copying large objects (performance improvement)
  - Allow functions to modify input arguments
  - Cleaner syntax compared to pointers (no * or &)

- **Important rule:**
  - A reference parameter behaves exactly like the original variable inside the function.
  - It is just another name for the same memory location.

8.4.2 References vs Pointers as Function Arguments

- In C++, there are two common ways to allow a function to modify an argument:
  - **References (C++)**
  - **Pointers (C/C++)**

- **Pointer-based arguments (C/C++ compatible):**

```
void foo(int* p) {
    *p = 10;
}

int main() {
    int x = 5;
    foo(&x);
}
```

  - The function receives the **address** of the variable.
  - You must explicitly pass `&x` (address-of operator).
  - Inside the function, you must dereference using `*p`.

- **Reference-based arguments (C++ only):**

```
void foo(int& x) {
    x = 10;
}

int main() {
    int x = 5;
    foo(x);
}
```

  - The function receives an **alias** to the original variable.
  - No need to use `&` or `*` at the call site.
  - The syntax is cleaner and behaves like the original variable.

- **Key difference in calling style:**

```
Pointer version:     foo(&x);
Reference version:   foo(x);
```

- **Conceptual difference:**
  - Pointer: function receives an **address** and manually dereferences it.
  - Reference: function receives an **alias** automatically (implicit dereference).

- **Important note:**
  - You do NOT write `foo(&arg)` when the function expects a reference.
  - `&` is only used when the function expects a pointer.

- **Listing 8.17**
- **Listing 8.18**
- **Listing 8.19**

8.4.3 Using const on References

- You might need to have **references** that are not allowed to change the **value**of the original variable being **aliased**.
- Using **const** when declaring such references is the way to achieve this:

```
int original = 30;
const int& constRef = original; // think of (const belongs to const (int) &, so can't change re
constRef = 40; // NOT OK! constRef can't change value in original
int& ref2 = constRef; // NOT OK! ref2 is not const
const int& constRef2 = constRef; // OK
```

8.4.4 Extended: Using const with References

- A **const reference** prevents modification of the value being referenced.

```
int original = 30;
const int& ref = original;

ref = 40;   // NOT OK: cannot modify through const reference
```

- A const reference still refers to the **same object**, but only allows read-only access.
- You can bind const references to const or non-const variables:

```
int x = 10;
const int& r1 = x;      // OK
const int& r2 = r1;     // OK
```

- You cannot modify the original through a const reference:

```
r1 = 20;   // NOT OK
```

- **Important rule:**
  - References in C++ are already fixed (cannot be reseated)
  - `const` only controls whether the value can be modified
  - `int& const` is meaningless in practice

```
int& -> fixed alias (cannot change what it refers to)
const int& -> fixed alias + cannot modify value
int& const -> conceptually meaningless / not used
```

8.4.5 Definition of Reference in C++ and why it's not valid in C

In C++, a *reference* is a language-level feature that creates an alias for an existing variable. Unlike pointers, references are not separate objects that store memory addresses; instead, they directly refer to the original variable.

- A reference must be initialized when it is declared.
- A reference cannot be reseated to refer to another variable after initialization.
- A reference behaves as an alias of the original variable.
- C does not support this language feature; it only provides pointers.

In C++, references are declared using the `&` symbol:

```
int a = 10;
int& ref = a;   // ref is a reference to a
```

This allows `ref` to be used exactly like `a`:

```
ref = 20;   // modifies a directly
```

In contrast, C does not have references as a built-in type system feature. Instead, it uses pointers to achieve similar indirect access to variables:

```
int a = 10;
int *p = &a;   // pointer to a
```

However, pointers differ fundamentally from C++ references:

- Pointers can be reassigned to point to different variables.
- Pointers require explicit dereferencing using `*`.
- Pointers store memory addresses explicitly, whereas references are aliases.

For example:

```
int a = 10;
int b = 20;

int* p = &a;
p = &b;   // valid: pointer reassignment

// In C++, a reference cannot be "re-pointed":
int& r = a;
// r = b;  // this assigns the value of b to a, not rebinding r
```

Thus, the key difference is that C++ provides true reference semantics as part of the language, while C only provides pointer-based indirect access, which is more flexible but less abstract.

8.4.6 QA

- **Q: Why dynamically allocated when you can do with static arrays where you don't need to worry about deallocation?**

  A: Static arrays have **fixed size** and will neither scale upward if your application needs more **memory** nor will they optimize your application needs less. Dynamic memory fixes that.

- **Q: I have 2 pointers:**

```
int* pointToAnInt = new int;
int* pCopy = pointToAnInt;
```

  Am I not better off calling 'delete' using both to ensure memory is gone?

  A: No. You are allowed to invoke `delete` only once on the **address** returned by `new`.

  Also, you would ideally **avoid** having two pointers pointing to the **same address** b/c performing `delete` on any one would **invalidate** the other.

  Your program should also not be written in a way that you have any uncertainity about the validity of **pointers** used.

- **Q: When should i use 'new(nothrow)'?**

  A: If you don't want to handle the **exception** `std::bad_alloc`, you use the `nothrow` version of **operator** `new` that returns the **NULL** if the requested allocation **fails**.

- **Q: I can call a function to calculate the area using the following two methods:**

```
void CalculateArea(const double* const ptrRadius, double* const ptrArea);
void CalculateArea(const double& radius, double& area);
```

  Which variant should I prefer?

  A: In general, prefer the **reference version** (C++):

```
void CalculateArea(const double& radius, double& area);
```

  Reason:

  - References provide **cleaner syntax** (no , no & at call site).
  - They guarantee a **valid object** must be passed (cannot be null).
  - They make the function easier to read and use.

  Important clarification:

  - It is not that pointers are always "invalid".
  - Pointers can be valid and safe if used correctly.
  - However, pointers can also be **null or uninitialized**, which introduces extra risk.

  Comparison:

  - Pointer version:
    - Can be null or invalid if misused

    - Requires explicit dereferencing (`*`)
    - More verbose at call site (`&x`)
  - Reference version:
    - Must always refer to a valid object
    - Cleaner syntax (acts like a normal variable)
    - Safer by default in most use cases

C vs C++ note:

- C does not support references.
- Therefore, C must use pointers.
- C++ supports both pointers and references.

- **Q: I have a pointer**

```
int number = 30;
const int* pointToAnInt = &number;
```

I understand that I cannot change the value of 'number' using the pointer 'pointToAnInt' due to the 'const' declaration. Can I assign 'pointToAnInt' to a non-const pointer and then use it to manipulate the value contained in integer number?

A: No, you cannot change the **const-correctness** of the pointer:

```
int* pAnother = pointToAnInt; // cannot assign pointer to const to a non-const
```

- **Q: What is the difference btw these 2 declarations?**

```
int myNums[100];
int* myArrays[100];
```

A: `myNums` is an array of integers — that is, `myNums` is a **pointer** to a **memory location** that holds 100 integers, points to the **first index** at index 0.

It is the static alternative to the following:

```
int* mNums = new int[100]; // dyanmically allocated array
delete[] myNums;
```

`myArrays`, on the other hand, is an array of 100 **pointers**, each pointer being capable of pointing to an integer or an array of integers.

8.4.7 Quiz

1. Why can't you assign a `const` reference to a `non-const` reference?

   A: You cannot bind a `const` reference to a `non-const` reference because it would allow the possibility of modifying a value that was originally declared as constant. This would violate C++'s type safety rules by potentially stripping away const-correctness. In general, C++ prevents dropping `const` qualifiers through reference assignment to ensure that immutable objects remain protected.

2. Are `new` and `delete` functions?

   A: No, `new` and `delete` are not functions. They are C++ language operators. Unlike functions, they are built into the language and can be overloaded, but they do not behave like normal function calls.

3. What is the nature of value contained in a **pointer variable**?

   A: A pointer variable stores a memory address as its value. This address refers to the location of another variable or object in memory.

4. What **operator** would you use to access the data pointed by a pointer?

   A: The dereference operator `*` is used to access the value stored at the memory address pointed to by a pointer.

9 Classes and Objects

- Thus far, we've learned ***procedural programming*** and haven't dived into ***object-oriented programming***. We now will.

9.1 Classes

- You declare a ***class*** by using the keyword `class` followed by the **name** of the class, followed by a **statement block** `{...}`
- A **declaration** of a **class** tells the **compiler** about the class and its properties.

- A class consists of its **fields** (data members) and ***methods*** (member functions).
- ***Encapsulation*** is the ability to logically **group data** and **functions** and we do that in our class in object-oriented programming.

9.1.1 An Object as an Instance of a Class

- A ***class*** is the **blueprint** and declaring a class alone has no interesting effect.

  The **real-world** avator of a class at program execution time is an ***object***.
- To use this blueprint you create an **instance** of the class, i.e. an ***object***.
- You use this **object** to access its **attributes** and **member methods**. (or just attributes/fields and methods/functions)

- Creating an **object** of type `class Human` is similar to creating an instance of another type, say `double`:

```
double pi = 3.14;
Human firstMan;  // statically allocated object
```

- Alternatively, you can **dynamically** create an **instance** of `class Human` using `new` as you would for another type say `int`:

```
int* pointToAnInt = new int;  // an integer allocated dynamically
delete pointsToAnInt;

Huamn* firstHuman = new Human(); // dynamically allocated Human
delte firstHuman;
```

9.1.2 Accessing Members Using the Dot Operator (.)

The dot operator (.) is used to access the data members and member functions of an object. It is applicable when working directly with an instance of a class rather than a pointer to that instance.

- Used with objects (instances) of a class.
- Accesses both data members and member functions.
- The object itself owns the members being accessed.

For example:

```
class Student {
public:
    std::string name;
    int age;

    void display() {
        std::cout << name << " is "
                  << age << " years old.\n";
    }
};

Student s;

s.name = "Alice";
s.age = 20;
s.display();
```

In this example, the object `s` accesses its data members and member function using the dot operator.

9.1.3 Accessing Members Using the Pointer Operator (->)

The pointer operator (->) is used to access the members of an object through a pointer. It combines pointer dereferencing and member access into a single operator.

- Used with pointers to objects.
- Accesses both data members and member functions.
- Equivalent to dereferencing the pointer and then using the dot operator:

  ```
  (*pointer).member
  ```

For example:

```
Student* p = new Student;

p->name = "Bob";
p->age = 22;
p->display();

delete p;
```

The above is equivalent to:

```
(*p).name = "Bob";
(*p).age = 22;
(*p).display();
```

The pointer operator is generally preferred because it is shorter, easier to read, and less prone to errors than explicitly dereferencing the pointer before using the dot operator.

- **Listing 9.1** A compile-worthy class Human

9.1.4 Class vs Struct

In C++, both `class` and `struct` can contain data members, member functions, constructors, destructors, and other class features. The primary difference between them is their default access level.

- Both `class` and `struct` can contain data members and member functions.
- Both support constructors, destructors, inheritance, and polymorphism.
- The default access specifier for a `class` is `private`.
- The default access specifier for a `struct` is `public`.
- The default inheritance mode for a `class` is `private` inheritance.
- The default inheritance mode for a `struct` is `public` inheritance.
- By convention, `struct` is typically used for simple data structures, while `class` is used for objects that encapsulate both data and behavior.

For example, the following class has private members by default:

```
class Student {
    int age;    // private by default
};
```

To make members accessible, the `public` access specifier must be used:

```
class Student {
public:
    std::string name;
    int age;
};
```

In contrast, a structure has public members by default:

```
struct Student {
    std::string name;   // public by default
    int age;
};
```

Thus, although `class` and `struct` are nearly identical in functionality, they differ primarily in their default access and inheritance rules, as well as their typical usage conventions.

9.1.5 Different Ways to Declare a `struct`

1. **Define the structure and declare an object immediately:**

```
struct Student {
    std::string name;
    int age;
} student1;
```

   Here, the structure type `Student` is defined, and an object named `student1` is created immediately.

2. **Define the structure first, then declare objects later (most common):**

```
struct Student {
    std::string name;
    int age;
};

Student student1;
Student student2;
```

   This is the preferred style because the type can be reused to create multiple objects.

3. **Declare multiple objects when defining the structure:**

```
struct Student {
    std::string name;
    int age;
} student1, student2, student3;
```

   Multiple objects can be declared immediately after the structure definition.

4. **Declare a pointer to a structure:**

```
struct Student {
    std::string name;
    int age;
};

Student* ptr = new Student;
```

   The pointer `ptr` stores the address of a dynamically allocated `Student` object.

5. **Anonymous structure (rarely used):**

```
struct {
    std::string name;
    int age;
} student1;
```

   This creates an object named `student1` whose type has no name. Since the structure is anonymous, additional objects of the same type cannot be declared later.

Note on `typedef struct` In the C programming language, structures are commonly declared using `typedef` so that the `struct` keyword does not need to be written when declaring variables.
For example:

```
typedef struct Student {
    char name[50];
    int age;
} Student;

Student s1;
```

It is also common to define an anonymous structure using `typedef`:

```
typedef struct {
    char name[50];
    int age;
} Student;

Student s1;
```

In C++, however, this is unnecessary because the structure name is automatically introduced as a type name:

```
struct Student {
    std::string name;
    int age;
};

Student s1;
```

Although `typedef struct` is valid C++, it is considered redundant and is rarely used in modern C++ code.

9.1.6 Public, Private, Protected, Friend in C++

Access specifiers in C++ control the visibility and accessibility of class members. They are essential for encapsulation and data hiding.

- **public**
  - Members declared as `public` are accessible from anywhere in the program.
  - They form the interface of the class.
- **private**
  - Members declared as `private` are only accessible within the same class.
  - They are used to hide implementation details.
- **protected**
  - Members declared as `protected` are accessible within the class and its derived (child) classes.
  - They are not accessible from outside the class hierarchy.
- **friend**
  - The `friend` keyword allows a non-member function or another class to access private and protected members of a class.
  - Friendship is granted explicitly and is not inherited or transitive.

Example:

```
#include <iostream>
using namespace std;

class Base {
private:
    int privateValue = 10;

protected:
    int protectedValue = 20;

public:
    int publicValue = 30;

    void show() {
        cout << privateValue << " "
             << protectedValue << " "
             << publicValue << endl;
    }

    friend void friendFunction(Base obj);
};

void friendFunction(Base obj) {
    // Can access private and protected members
    cout << obj.privateValue << endl;
    cout << obj.protectedValue << endl;
    cout << obj.publicValue << endl;
}

class Derived : public Base {
public:
    void access() {
```

```
        // cout << privateValue;   // not accessible
        cout << protectedValue << endl; // accessible
        cout << publicValue << endl;    // accessible
    }
};

int main() {
    Base b;
    b.publicValue = 100;

    b.show();
    friendFunction(b);

    Derived d;
    d.access();

    return 0;
}
```

Key Summary:

- `public` → accessible everywhere
- `private` → accessible only inside the class
- `protected` → accessible in class and derived classes
- `friend` → grants special access to external functions/classes

9.1.7 This Keyword

The `this` keyword in C++ is an implicit pointer available inside all non-static member functions. It points to the object that invoked the member function.

- `this` is a pointer to the current object.
- It is automatically passed to all non-static member functions.
- It is used to access members of the current object explicitly.
- It helps resolve ambiguity when parameter names shadow member variables.
- It cannot be used in static member functions.

Example:

```
#include <iostream>
using namespace std;

class Student {
private:
    int age;

public:
    void setAge(int age) {
        // 'this->age' refers to the member variable
        // 'age' refers to the parameter
        this->age = age;
    }

    void display() {
        cout << "Age: " << this->age << endl;
    }
};

int main() {
    Student s;
    s.setAge(20);
    s.display();

    return 0;
}
```

Key Points:

- `this` is an implicit pointer of type `ClassName* const`.
- It always points to the current object instance.
- It is commonly used to disambiguate member variables from parameters.
- It improves readability in method chaining scenarios.

9.1.8 Static Variables in Classes

A `static` data member in a class is shared among all objects of that class rather than each object having its own separate copy.

- A `static` member variable belongs to the class, not individual objects.
- Only one copy of the variable exists, regardless of how many objects are created.
- It is initialized outside the class definition.
- It can be accessed using the class name or through an object.
- It retains its value throughout the lifetime of the program.

Example:

```
#include <iostream>
using namespace std;

class Counter {
public:
    static int count;

    Counter() {
        count++;
    }

    void display() {
        cout << "Count: " << count << endl;
    }
};

// Definition and initialization of static member
int Counter::count = 0;

int main() {
    Counter c1;
    Counter c2;
    Counter c3;

    c1.display();
    c2.display();
    c3.display();

    cout << "Final count: " << Counter::count << endl;

    return 0;
}
```

Key Points:

- Static members are shared across all instances of a class.
- They must be defined outside the class using scope resolution operator `::`.
- They are useful for tracking shared state (e.g., object count, configuration values).

9.2 Constructors

- A ***constructor*** is a **special function** (or method) invoked durinkg the **instantiation** of a **class** to construct an **object**.

9.2.1 Declaring and Implementing a Constructor

- A **constructor** is a **special function** that takes the name of the class and returns **no value**.
- So, `class Human` would have a constructor that is declared like this:

```
class Human
{
public:
 Human(); // declaration of a constructor
}
```

- This constructor can be implemented either **inline** within the class or **externally** outside the class declaratiopn.
- An ***implementation*** (also called ***definition***) inside the class looks like this:

```
class Human
{
public:
 Human()
 {
  // constructor code here
 }
};
```

- A variant enabling you to define the constructor **outside** the class's declaration looks like this:

```
class Human
{
public:
 Human(); // constructor declaration
};

// constructor implementation (definition)
Human::Human()
{
 // constructor code here
}
```

- the `::` is called the ***scope resolution operator***.

  For example, `Human::dateOfBirth` is referring to the **variable** `dateOfBirth` declared **within the scope** of class `Human`.

- `::dateOfBirth` would refer to the `dateOfBirth` in the **global scope**.

9.2.2 When and How to Use Constructors

- An **constructor** is always invoked during **object creation**, when an **instance** of a class is constructed.
- This is a perfect time to initialize the class members of your instance.
- **Listing 9.3** Using Constructors

9.2.3 Overloading Constructors

- Constructors can be **overloaded** (mean multiple constructors with different parameters) just like functions.
- Example:

```
class Human
{
public:
 Human() {...} // default constructor code

 Human(string humansName) {...} // overloaded constructor code
};
```

- **Listing 9.4** Multiple Constructors
- You can choose to **not** implement the ***default constructor*** to enforce **object instantiation** with certain minimal parameters as explained in the next section.

9.2.4 Class Without a Default Constructor

- **Listing 9.4** Class with overload constructor and No Default Constructor
- You also need not to declare any constructor since the default one is always provided implicitly

9.2.5 Constructor Parameters with Default Values

- Constructors can have **default values** like functions, allowing objects to be created with fewer arguments while still initializing members safely.

```
#include <iostream>
#include <string>

using namespace std;

class Student {
private:
    string name;
    int age;
    double gpa;

public:
    // Constructor with default parameter values
    Student(string n = "Unknown", int a = 0, double g = 0.0) {
        name = n;
        age = a;
        gpa = g;
    }

    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "GPA: " << gpa << endl;
    }
};

int main() {
    Student s1;                      // uses all defaults
    Student s2("Alice");            // only name provided
    Student s3("Bob", 22);         // name + age
    Student s4("Charlie", 21, 3.9);// full initialization

    s1.display();
    cout << "-----------------" << endl;

    s2.display();
    cout << "-----------------" << endl;

    s3.display();
    cout << "-----------------" << endl;

    s4.display();

    return 0;
}
```

9.2.6 Constructors with Initialization Lists

An initialization list initializes a class's data members before the constructor body is executed. This is the preferred method of initializing member variables in modern C++.

- An initialization list appears after the constructor parameter list and before the constructor body.
- Data members are initialized directly rather than first being default-constructed and then assigned values.
- Initialization lists are generally more efficient than assigning values inside the constructor body.
- They are **required** when initializing:
  - `const` data members.
  - Reference (`&`) data members.
  - Class members that do not have a default constructor.
- Data members are initialized in the order they are declared in the class, **not** in the order they appear in the initialization list.

Syntax:

```
ClassName(type parameter)
    : member(parameter)
{
    // Constructor body
}
```

Example:

```
#include <iostream>
#include <string>

using namespace std;

class Student {
private:
    string name;
    int age;
    double gpa;

public:
    Student(string n, int a, double g)
        : name(n), age(a), gpa(g)
    {
    }

    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "GPA: " << gpa << endl;
    }
};

int main() {
    Student s1("Alice", 20, 3.8);
    s1.display();

    return 0;
}
```

Example Requiring an Initialization List:

```
#include <iostream>

using namespace std;

class Example {
private:
    const int id;
    int& value;

public:
    Example(int i, int& v)
        : id(i), value(v)
    {
    }

    void display() {
        cout << "ID: " << id << endl;
        cout << "Value: " << value << endl;
    }
};

int main() {
    int x = 100;

    Example obj(1, x);
    obj.display();

    return 0;
}
```

In the second example, the initialization list is mandatory because both `id` (a `const` member) and `value` (a reference member) must be initialized when the object is created. They cannot be assigned values inside the constructor body.

9.3 Destructor

- A ***destructor***, like a **constructor**, is a **special function**.
- A **constructor** is invoked at **object instantiation**, and a **destructor** is automatically invoked when an **object** is **destroyed**.

9.3.1 Declaring and Implementing a Destructor

- The **destructor** looks like a **function** that takes the name of the **class**, yet has a **tilde (~)**.
- Example:

```
class Human
{
public:
 ~Huamn(); // declaration of a destructor
};
```

- The **destructor** can be implemented **inline** in the class or **externally** outside the class declaration.

internally:

```
class Human
{
public:
 ~Human()
 {
  // destructor code here
 }
};
```

externally:

```
class Human
{
public:
 ~Human(); // destructor declaration
};

// destructor defintion (implementation)
Human::~Human()
{
 // destructor code here
}
```

9.3.2 When and How to Use a Destructor

- A **destructor** is always invoked when an **object** of a class is **destroyed** when it **goes out of scope** or is **deleted** via `delete` or `delete[]` or free, etc.
- This property makes a **destructor** the ideal place to reset variables and **release dynamically allocated memory** and other resources.
  - `std::string` automatically manages dynamic memory for its character data, relieving the programmer from manually allocating and deallocating memory.
  - The `std::string` object itself is stored wherever it is created (e.g., on the stack for local variables or on the heap if dynamically allocated).
  - The character data managed by `std::string` may be stored within the object itself (using Small String Optimization) or in dynamically allocated heap memory, depending on the implementation and the length of the string.
  - But internally it would use constructors and destructors and operators.

- **Listing 9.7** Destructor
- Smart pointers (Lesson 26) is where destructors play a critical role in working with pointers in a smarter way.
- **Destructors** cannot be overloaded.
- A class can only have **one destructor**, the compiler creates and invokes a dummy destructor, that is, an empty one (that does no cleanup of dynamically allocated memory)

9.4 Copy Constructors

- Argument passing to functions copy the argument into the function stack frame whenevery the function is called and of course if it has arguments passed to it.
- This rule applies to **objects**, that is, instances of classes as well.

9.4.1 Shallow Copying and Associated Problems

- Classes such as `MyString` in **Listing 9.7** had a **pointer member** `buffer` that points to **dynamically allocated memory**, allocated in the constructor using `new` and then deallocated in the destructor using `delete[]`.
- When an **object** of this class is **copied**, the **pointer member** is **copied**, but not the **pointer memory**, resulting in 2 objects pointing to the **same dynamically allocated buffer in memory**.

  So,

  - Classes such as `MyString` in **Listing 9.7** contain a pointer member (`buffer`) that points to dynamically allocated memory. The memory is allocated in the constructor using `new` and released in the destructor using `delete[]`.
  - When an object is copied, the compiler performs a **shallow copy** by default. This copies the **pointer value** (memory address), but **does not allocate or copy the dynamically allocated memory** it points to.
  - As a result, both objects' pointer members refer to the **same block of dynamically allocated memory**. If one object modifies or deallocates the memory, it affects the other object, potentially leading to errors such as double deletion.

- Hence, when an **object** is **destructed**, `delete[]` deallocates the memory, thereby **invalidating** the **pointer copy** held by the other object.
- Such copies are ***shallow*** and are a **threat** to the stability of the program, as **Listing 9.8** Shallow copy Problem

Here is another small example:

```
#include <iostream>
#include <cstring>

using namespace std;

class MyString {
private:
    char* buffer;

public:
    MyString(const char* str) {
        buffer = new char[strlen(str) + 1];
        strcpy(buffer, str);
    }

    ~MyString() {
        delete[] buffer;
    }

    void display() {
        cout << buffer << endl;
    }
};

int main() {
    MyString s1("Hello");

    // Default copy constructor performs a shallow copy
    MyString s2 = s1;

    s1.display();
    s2.display();

    // When main() ends:
    // ~MyString() is called for s2 -> delete[] buffer;
    // ~MyString() is called for s1 -> delete[] buffer; (same memory!)
    // Result: Undefined behavior (double deletion)

    return 0;
}
```

This is a classic example demonstrating why the default copy constructor can be dangerous for classes that own dynamically allocated memory, and why a deep copy (custom copy constructor) is needed.

Output for listing 9.8:

```
String buffer in sayHello is 23 characters long
Buffer contains: Hello from String Class
Invoking destructor, clearing up for id: 21
Invoking destructor, clearing up for id: 0
free(): double free detected in tcache 2
Aborted                    (core dumped) ./a.out
```

where i changed the id in the callee to 21 and that shows the passed arg first destructs;

- So, the issue is both objects point to the same memory address, but when they both destructor we get the error of freeing an already deleted/freed data

9.4.2 Ensuring Deep Copy Using a Copy Constructor

- The **copy constructor** is an **overloaded constructor** that you supply.
- It is invoked everytime an **object** of the class is **copied**.
- So, we shall write a ***copy constructor*** to help solve the issues with **shallowing copying**, which we call ***deep copying***.

- The **declaration syntax** of a **copy constructor** for `class MyString` is the following:

```
class MyString
{
 MyString(const MyString& copySource); // copy constructor
 // the const says "I promise not to modify the object I'm copying from."
};

another reason for the const:
MyString::MyString(const MyString& copySource) {
    // OK: read from copySource
    std::cout << copySource.buffer << std::endl;

    // Error: cannot modify the source object
    // copySource.buffer = nullptr;
}

...

MyString::MyString(const MyString& copySource)
{
 // copy constructor implementation code
}
```

- Thus, a ***copy constructor*** takes an **object** of the **same class** by **reference** as a parameter.
- This parameter is an **alias** of the **source object** and is the handle you have in writing your **custom copy code**.
- You would use the **copy constructor** to **ensure** a ***deep copy*** of **all buffers** in the source.

  Like before you would only just copy the memory address, but here you would **allocate new memory on the heap** and then **copy the actual data contents** from the source object into this newly allocated memory, so that **both** objects **own independent copies** of the **data** rather than sharing the **same memory address**.
- **Listing 9.9** Define a copy constructor to ensure deep copy of dynamically allocated buffers.

- The **syntax difference** between a **regular constructor** and a **copy constructor** is in their **parameter lists**.

  A regular constructor takes **raw values** (e.g., integers, strings, pointers), while a copy constructor takes a **reference to another object of the same class**.

```
// Regular constructor
MyString(const char* str);

// Copy constructor
MyString(const MyString& other);
```

- The copy constructor is used to create a new object from an existing object of the same type, typically performing a **deep copy** of dynamically allocated resources.
- Yes, the **copy constructor is invoked implicitly** in several situations where an object is copied. These include:
  - When an object is initialized from another object:

    ```
    MyString s2 = s1;
    ```

  - When an object is passed to a function by value:

    ```
    void func(MyString s);
    func(s1);
    ```

  - When an object is returned by value from a function:

    ```
    MyString func();
    ```

- If no copy constructor is defined, C++ generates a default one that performs a **shallow copy**, meaning it copies pointer values rather than the data they point to.
- A **copy constructor** can **only take one argument**, which is a **reference to an object** of the **same class** (typically `const ClassName&`).

  This is because its purpose is to create a **new object as a copy** of an **existing object**.
- If more than one parameter is used, it is no longer considered a copy constructor but instead a regular overloaded constructor.
- The standard form is:

  ```
  ClassName(const ClassName& source);
  ```

- The compiler does not "guess" or manually pass the object. Instead, when a copy constructor is invoked, the compiler automatically selects it based on the context where a copy is required (e.g., initialization from another object, passing by value, or returning by value).

  Internally, the compiler rewrites the code so that the source object is passed as the single argument to the copy constructor. For example:

  ```
  MyString s2 = s1;
  ```

  is conceptually transformed by the compiler into:

  ```
  MyString s2(s1);
  ```

  Here, `s1` is explicitly passed as the argument to the copy constructor. The "implicit" part refers to the fact that the programmer does not explicitly call the constructor; the compiler inserts the call automatically based on the context.

- The **copy constructor** has ensured a **deep copy** in cases such as:

```
MyString sayHello("Hello");
UseMyString(sayHello);
```

- However, what if you tried copying via **assignment**:

```
MyString overwrite("who cares? ");
overwrite = sayHello;
```

  This would **still** be a **shallow copy** because you still haven't yet supplied a ***copy assignment operator=***.

- In the absence of one, the compiler has supplied a default for you that does a **shallow copy**.
- The ***copy assignment operator*** is discussed in Lesson 12. Listing 12.8 is an improvied `MyString` that implements the same:

```
MyString::operator= (const MyString& copySource)
{
 // ... copy assignment operator code
}
```

- Using `const` in the **copy constructor declaration** ensures that the **copy constructor** does **not modifiy** the **source object** being referred to.
- Additionally, the parameter in the **copy constructor** is **passed by reference** as a necessity.

  If it weren't a reference, the **copy constructor** would itself invoke a copy, thus invoking itself again and so on till the system runs out of memory.

- Using `const` in the copy constructor ensures that the source object cannot be modified during copying. This guarantees that the original object remains unchanged and allows copying from both `const` and non-`const` objects.
- The copy constructor parameter must be passed by reference. If it were passed by value, a copy would need to be made to pass the argument into the constructor. This would recursively invoke the copy constructor again, leading to infinite recursion and eventually a stack overflow.
- Therefore, the correct and standard form of a copy constructor is:

```
ClassName(const ClassName& other);
```

- **Do** always implement a **copy constructor** and **copy assignment operator** when your class contains **raw pointer members** (e.g. `char*`). This prevents shallow copying and avoids issues such as double deletion and dangling pointers.
- **Do** declare the copy constructor using a **const reference parameter**:

  ```
  ClassName(const ClassName& other);
  ```

  This ensures the source object is not modified during copying and allows copying from both const and non-const objects.
- **Do** use the `explicit` keyword for constructors that take a single parameter to prevent unintended implicit conversions. For example:

  ```
  class MyString {
  public:
      explicit MyString(const char* str);
  };

  MyString s1 = "Hello";   // Not allowed (prevents implicit conversion)
  MyString s2("Hello");    // Allowed (explicit construction)
  ```

- **Do** prefer using standard library types such as `std::string` and smart pointers (e.g. `std::unique_ptr`, `std::shared_ptr`) as class members instead of raw pointers, since they automatically manage memory and implement safe copy/move semantics.
- **Don't** use raw pointers as class members unless absolutely necessary, as they require manual memory management and increase the risk of memory leaks, double deletion, and undefined behavior.

9.4.3 Explicit keyword

- The `explicit` keyword in C++ is used to **prevent implicit conversions** that the compiler would otherwise perform automatically using constructors.
- Without `explicit`, a constructor that takes a single parameter can be used for implicit type conversion. For example:

```
class MyString {
public:
    MyString(const char* str) {
        // constructor
    }
};

MyString s = "Hello";   // implicit conversion allowed
```

- In this case, the compiler automatically converts the string literal into a `MyString` object.
- When the constructor is marked as `explicit`, implicit conversions are disallowed:

```
class MyString {
public:
    explicit MyString(const char* str) {
        // constructor
    }
};

MyString s = "Hello";   // Error: implicit conversion not allowed
MyString s2("Hello");   // OK: explicit construction
```

- The `explicit` keyword is used to prevent unintended or accidental conversions that may lead to bugs or unclear code.
- In summary, `explicit` forces the programmer to clearly state when a conversion or constructor call should occur.
- ***explicit*** is a **compile-time** rule that **prevents the compiler** from performing **automatic (implicit) type conversions** using constructors.

9.4.4 Move Constructors Help Improve Performance

- There are cases where **objects** are subjected to copy steps automatically, due to the nature of the language and its needs.
- Consider:

```
class MyString
{
 // pick implementation from listing 9.9
};

MyString Copy(MyString& source)  // function
{
 MyString copyForReturn(source.GetString());  // create copy
 return copyForReturn; // return by value invokes copy constructor
}

int main()
{
 MyString sayHello("Hi");
 MyString sayHelloAgain(Copy(sayHello));  // invokes 2x copy constructor
 return 0;
}
```

- As the comment indicates, in the **instantiation** of `sayHelloAgain`, the **copy constructor** is invoked twice, thus a **deep copy** was performed **twice** b/c our call to function `Copy(sayHello)` returns a `MyString` **by value**.
- However, this value returned is **very temporary** and is **not avaiable** outside this expression.

  So, the **copy constructor** invoked in good faith by the C++ compiler is burden on **performance**.

  This impact becomes significant if our class were to contain objects of great size.

- To **avoid** this performance bottleneck, versions of C++ starting with C++11 feature a ***move constructor*** in addition to a **copy constructor**.
- The syntax of a ***move constructor*** is

```
// move constructor
MyString(MyString&& moveSource)
{
 if (moveSource != NULL)
 {
  buffer = moveSource.buffer;  // take ownership i.e. 'move'
  moveSource.buffer = NULL;  // set the move source to NULL
 }
}
```

  - The constructor

    `MyString(MyString&& moveSource)`

    is called a **move constructor**.

– Here's what each part means:
  * `MyString` — the constructor's name.
  * `MyString&&` — an **rvalue reference** to another `MyString`.
  * `moveSource` — the parameter name.
– In other words, this constructor says:

  "This constructor takes ownership of another temporary (or movable) `MyString`."
– Compare the different constructor signatures:

```
MyString(MyString other);       // pass by value
MyString(MyString& other);      // lvalue reference
MyString(MyString&& other);     // rvalue reference
```

– They each accept different kinds of arguments.
– Normally, variables are **lvalues**. For example:

```
MyString a("hello");
```

  Since `a` has a name, it is an lvalue.
– A temporary object is an **rvalue**. For example:

```
MyString("hello")
```

  or

```
return MyString("hello");
```

  These objects are about to disappear, so C++ allows their resources to be transferred instead of copied.
– Example:

```
MyString a("hello");
MyString b(std::move(a));   // calls move constructor
```

  `std::move(a)` converts `a` into an rvalue, so the move constructor is selected:

```
MyString(MyString&& moveSource)
```

– Inside the move constructor, the implementation may look like:

```
MyString::MyString(MyString&& moveSource)
{
    data = moveSource.data;      // steal pointer
    moveSource.data = nullptr;   // leave source safe
}
```

  No new memory is allocated. The pointer ownership is simply transferred.
– Why not use `MyString&`?
  If the constructor were

```
MyString(MyString& other)
```

  then it could receive a normal object that the caller still intends to use. Stealing its resources would unexpectedly leave that object in an unusable state.
– The `&&` tells the compiler:

  "This object is temporary (or has been explicitly marked movable), so it is safe to transfer its resources."
– Summary:
  * `&` = reference to a normal object (do not steal its resources).
  * `&&` = reference to a temporary or movable object (safe to steal its resources).
  * `MyString(MyString&& moveSource)` is the constructor that performs a **move** instead of a copy.

- When a **move constructor** is programmed, the compiler automatically opts for the same for "moving" the temporary resource and hence **avoiding** a **deep-copy** step.
- With this **move consutrctor** implemented, the comment should be appropriately changed to the following:

```
MyString sayHelloAgain(Copy(sayHello)); // invokes 1x copy, 1x move constructors
```

- The ***move constructor*** is usually implemented with the ***move assigment operator***, which is dicussed in Lesson 12. Listing 12.11 is a better MyString that implements the **move constructor** and the **move assignment operator**.

9.5 Different Uses of Constructors and Destructors

- We've learned now concepts that enable you to create classes that can control how they're **created, copied, destroyed,** or **expose data (through** ***access specificers***).

9.5.1 Class That Does Not Permit Copying

- Somtimes we don't want duplicates at all in our program:

```
President ourPresident;
DoSomething(ourPresident);  // duplicate created in passing by value
President clone;
clone = curPresident;   // duplicate via assignment
```

- You need to ensure that certain resources **cannot** be **copied** or **duplicated**.
- If you **don't** declare a **copy constructor**, the C++ compiler inserts a **default public copy constructor** for you.
- This ruins your design and threatens your implementation.
- We have a solution.
- You would ensure that your `class` **cannot be copied** by declaring a `private` **copy constructor**.
- This ensures that the function call `DoSomething(ourPresident)` will cause a compiler failure.
- To avoid **assignment**, you declare a `private` **assignment operator**.

- Thus, the solution is the following:

```
class President
{
private:
 President(const President&);    // private copy constructor
 President& operator= (const President&); // private copy assignment operator

 // ... other attributes
};
```

- There is **no need for implementation** of the **private copy constructor** or **assignment operator**.
- Just **declaring them private** is adequate and **sufficient** toward fulfilling your goal of ensuring **non-copyable objects** of `class President`.

9.5.2 Singleton Class That Permits a Single Instance

- `class President` earlier is good, but has a **shortcoming**:
  It **cannot help** creation of multiple presidents via instantiation of **multiple objects**:

  ```
  President One, Two, Three;
  ```

- Individually, they are **non-copyable** thanks to the **private copy constructors**, but what youi ideally need is a `class President` that has **one**, and only one, real-world manifestation — that is, there is **only one object** and creation of additional ones is prohibited.

- Welcome to the concept of ***singleton*** that uses ***private constructors***, a ***private assignment operator***, and a ***static instance member*** to create this (controversally) powerful pattern.

- When the keyword ***static*** is used on a **class's data member**, it ensures that the member is **shared** across **all instances**
- When ***static*** is used on a **local variable** declared within the scope of a function, it ensures that the **variable retains its value** between **functions calls**.
- When ***static*** is used on a **member function** — a **method** — the method is **shared** across **all instances** of the class.

- **Listing 9.10** Singleton class President

9.6 Class That Prohibts Instantiation on the Stack

- Space on the stack is often limited
- If you are writing a database that may contain **terabytes** of data in its internal structures, you might want to ensure that a client of this **class** **cannot instantiate** it on the **stack**;
  instead it is **forced** to create **instances** only on the free store.
- The key to ensuring this is **declaring** the **destructor** `private`:

```
class MonsterDB
{
private:
 ~MonsterDB();  // private destructor
 // ... members that consume a huge amount of data
};
```

- **Declaring** a **private destructor** ensures that one is **not allowed** to create an **instance** like this:

```
int main()
{
 MonsterDB myDatabase; // compile error
 // ... more code
 return 0;
}
```

- This **instance**, if successfully constructed, would be on the **stack**.
- **All objects** on the **stack** get popped when the **stack** is unwound and therefore the compiler would need to compile and invoke the **destructor** `~MonsterDB()` at the **end** of `main()`.
- However, this **destructor** is **private** and there **inaccessible**, resulting in a compile failure.

- A **private destructor** would **not stop** you from instantiating on the ***heap***:

```
int main()
{
 MonsterDB* myDatabase = new MonsterDB(); // no error
 // ... more code
 return 0;
}
```

- We have a memory leak here: as the **destructor** is **not accessible** from **main**, you **cannot** do a `delete`, either.
- What `class MonsterDB` needs to support is a `public static` **member function** that would **destroy** the **instance** (a class member would have access to the **private destructor**.)
- **Listing 9.11** Only allowed creation on heap

9.6.1 Using Constructors to Convert Types

- Earlier we've learned **constructors** can be **overloaded**, that is, they may take **one or more parameters**.
- This feature is often used to **convert** one type to another.
- Let's see a `class Human` that features an **overloaded constructor** that accepts an integer.

```
class Human
{
 int age;
public:
 Human(int humansAge) : age(humansAge) {}
};


// function that takea  human as a parameter
void DoSomething(Human person)
{
 cout << "Human sent did something" << endl;
 return;
}
```

- The **constructor** allows a **conversion**:

```
Human kid(10); // convert integer into a Human
DoSomething(kid);
```

- I think the textbook messed here/is misleading so here is update:

```
No conversion (explicit construction)
Human kid(10);
DoSomething(kid);
```

  - Yes. If you write:

    ```
    Human kid(10);
    DoSomething(10);
    ```

    the `Human` object named `kid` is **not** used.
  - Instead, the compiler creates a **completely separate temporary** `Human` object for the function call.
  - Conceptually, it is as if you had written:

    ```
    Human kid(10);      // Your object

    Human temp(10);     // Compiler-created temporary
    DoSomething(temp);
    ```

  - Therefore, there are **two different** `Human` objects:
    * `kid` — the object that you explicitly created.
    * `temp` — a temporary object automatically created by the compiler.
  - If, instead, you write:

```
Human kid(10);
DoSomething(kid);
```

then there is **no type conversion**. The function receives the object `kid` (or a copy of it, since the parameter is passed by value).

- Compare the two cases.
  - **Case 1: No conversion**

    ```
    Human kid(10);
    DoSomething(kid);
    ```

    The object flow is:

    ```
    kid  ------------------> DoSomething
    ```

  - **Case 2: Implicit conversion**

    ```
    DoSomething(10);
    ```

    The object flow is:

    ```
    10
     |
     v
    Human(10)   <-- temporary object created by the compiler
     |
     v
    DoSomething
    ```

- Therefore, the temporary object is **not** the same object that you previously created. It is a brand new object that exists only for the duration of the function call and is destroyed immediately afterward.
- This automatic creation of a temporary object is what allows a **single-argument constructor** to perform an **implicit type conversion**.

- We fix this via the keyword ***explicit***

- Such **converting constructors** allow ***implicit conversions***.

```
Human anotherKid = 11; // int converted to Human
DoSomething(10);  // 10 converted to Human!
```

- We declared `DoSomething(Human person)` as a **function** that accepts a parameter of **type** `Human` and does not accept an `int`!
- So why did the line work?
- The compiler knows that `class Human` **supports** a **constructor** that accepts an integer and performed **implicit conversion**!

  It created an **object** of type `Human` using the integer you supplied and sent it as an argument to the function.

- To **void implicit conversions**, use keyword ***explicit*** at the time of **declaring the constructor**.

```
class Human
{
 int age;
public:
 explicit Human(int humansAge) : age(humansAge) {}
};
```

- Using `explicit` is **not** a prerequisite but in many cases a **good programming practice**.
- **Listing 9.12** Using explicit to block implicit conversions

- The problem of **implicit conversion** and **avoiding** them using keyword ***explicit*** applies to **operators** too.

9.7 sizeof() a Class

- The `sizeof` operator returns the number of **bytes** occupied by an object or a type in memory.
- When applied to a class, `sizeof` returns the total amount of memory required to store all of its **data members**, including any **padding bytes** that the compiler inserts for memory alignment.
- The size of a class **does not** include the code for its member functions. Member functions are stored only once in the program's executable, regardless of how many objects of the class are created.
- A class may contain:
  - Fundamental data types (such as `int`, `char`, and `double`).
  - Objects of other classes.
  - Structures (`struct`).
  - Arrays and pointers.
- When a class contains another class or a structure as a data member, the size of the containing class includes the size of those objects as well.
- Consider the following example:

```
struct Address
{
    int houseNumber;
    char streetInitial;
};

class Birthday
{
public:
    int day;
    int month;
    int year;
};

class Student
{
private:
    int id;             // Fundamental type
    double gpa;         // Fundamental type
    char grade;         // Fundamental type

    Address address;    // Structure
    Birthday birth;     // Another class
};
```

- The size of a `Student` object is conceptually the sum of the sizes of all its data members:

```
sizeof(Student)
    =
sizeof(int)
+ sizeof(double)
+ sizeof(char)
+ sizeof(Address)
+ sizeof(Birthday)
+ padding (if required)
```

- The compiler may insert **padding bytes** between members so that each data member begins at an address suitable for efficient access by the CPU.
- Therefore, the value returned by `sizeof(Student)` is often **larger** than simply adding the sizes of the individual data members.
- For example:

```
#include <iostream>
using namespace std;

int main()
{
    cout << "sizeof(Address)  = "
         << sizeof(Address) << endl;

    cout << "sizeof(Birthday) = "
         << sizeof(Birthday) << endl;

    cout << "sizeof(Student)  = "
         << sizeof(Student) << endl;

    return 0;
}
```

- The exact output depends on:
  - The compiler being used.
  - The target architecture (32-bit or 64-bit).
  - The alignment rules of the system.
- In general,

```
sizeof(class) =
sum of the sizes of its data members
+ any compiler-added padding
```

- Remember that `sizeof` measures only the memory occupied by the object's data. It does **not** include memory used by member functions, nor does it include memory dynamically allocated through pointers.

So:

```
Typical output (64-bit system):

sizeof(Address)  = 8
sizeof(Birthday) = 12
sizeof(Student)  = 40
```

Explanation of the output

- Assume typical sizes:
  - `int` = 4 bytes
  - `double` = 8 bytes
  - `char` = 1 byte
- **Address struct:**

```
struct Address
{
    int houseNumber;      // 4 bytes
    char streetInitial;   // 1 byte
};
```

Memory layout:

```
houseNumber     4 bytes
streetInitial   1 byte
padding         3 bytes
-----------------------
Total           8 bytes
```

- **Birthday class:**

```
class Birthday
{
public:
    int day;
    int month;
    int year;
};
```

Memory layout:

```
day     4 bytes
month   4 bytes
year    4 bytes
----------------
Total   12 bytes
```

- **Student class:**

```
class Student
{
private:
    int id;             // 4 bytes
    double gpa;         // 8 bytes
    char grade;         // 1 byte

    Address address;    // 8 bytes
    Birthday birth;     // 12 bytes
};
```

Memory layout (typical alignment):

```
id              4 bytes
padding         4 bytes   // align double

gpa             8 bytes

grade           1 byte
padding         7 bytes   // align Address

address         8 bytes

birth           12 bytes

padding         4 bytes   // final alignment
-----------------------------------------------
Total          40 bytes
```

- Even though the raw sum of members is:

  ```
  4 + 8 + 1 + 8 + 12 = 33 bytes
  ```

- The extra bytes come from **padding added by the compiler** to satisfy memory alignment rules, which makes access faster on most CPUs.

9.8 How struct differs from class

- struct are public by default.
- class is private by default.
- struct implementation:

```
#include <iostream>
#include <string>

using namespace std;

struct Student {
private:
    string name;
    int age;
    double gpa;

public:
    // default constructor
    Student() {}

    // Constructor to initialize data members
    Student(string n, int a, double g) {
        name = n;
        age = a;
        gpa = g;
    }

    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "GPA: " << gpa << endl;
    }
};

int main() {
    // Creating objects using constructor
    Student s1("Alice", 20, 3.8);
    Student s2("Bob", 22, 3.5);

    s1.display();
    cout << "------------------" << endl;
    s2.display();

    return 0;
}
```

- class implementation

```
#include <iostream>
#include <string>

using namespace std;

class Student {
private:
    string name;
    int age;
    double gpa;

public:
 Student() {} // default constructor

    // Constructor to initialize data members
    Student(string n, int a, double g) {
        name = n;
        age = a;
        gpa = g;
    }

    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "GPA: " << gpa << endl;
    }
};

int main() {
    // Creating objects using constructor
    Student s1("Alice", 20, 3.8);
    Student s2("Bob", 22, 3.5);

    s1.display();
    cout << "-----------------" << endl;
    s2.display();

    return 0;
}
```

- Lookover

Class vs Struct

section above for more details.

9.9 Declaring a 'friend' of a class

- A class **does not permit** access to its **data members and methods** that are declared **private**.
- The rule is waived for **classes** and **functions** that are disclosed as ***friend classes*** or ***friend functions*** using the keyword ***friend***
- **Listing 9.14** friend

```
class B;

class A
{
private:
    int secret = 99;

    friend class B;
};

class B
{
public:
    void show(A obj)
    {
        std::cout << obj.secret << std::endl; // allowed
    }
};
```

9.10 union: A Special Data Storage Mechanism

- A ***union*** is a special class type where all non-static data members share the same memory location.
- This means only one member can hold a meaningful value at a time.
- A union **cannot participate in inheritance hierarchies**. It cannot be used as a base class or derived class.
- The `sizeof` a union is always equal to the size of its **largest member**, even if that member is not currently active in a given instance.
- Example:

```
#include <iostream>
using namespace std;

union Data
{
    int i;
    float f;
    char c;
};

int main()
{
    Data d;

    d.i = 42;
    cout << "i = " << d.i << endl;

    d.f = 3.14f;
    cout << "f = " << d.f << endl;

    d.c = 'A';
    cout << "c = " << d.c << endl;

    cout << "sizeof(Data) = " << sizeof(Data) << endl;

    return 0;
}
```

- In this example, all members share the same memory, so writing to one member **overwrites** the **previous value**.
- Even though multiple members exist, only one is valid at any time.
- The output of `sizeof(Data)` depends on the largest member:
  - `int` might be 4 bytes
  - `float` might be 4 bytes
  - `char` might be 1 byte
- Therefore:

```
sizeof(Data) = size of largest member (usually 4 bytes in this example)
```

- The fact that a larger member exists does not change the size unless alignment/padding rules require it.
- C++17 introduces `std::variant` as an alternative typesafe structure to `union`

9.11 Using Aggregate Initialization on Classes and Structs

- ***Aggregate initialization*** is a way of initializing objects (typically `struct`s and simple `class`es) using curly braces `{}` without explicitly calling a constructor.
- It allows you to initialize all public data members in order.
- This works best with `struct`s or classes that behave like simple data containers (no private members or user-defined constructors in older rules).
- Example using a `struct`:

```
#include <iostream>
using namespace std;

struct Point
{
    int x;
    int y;
};

int main()
{
    Point p1 = {10, 20};   // aggregate initialization

    cout << "x = " << p1.x << endl;
    cout << "y = " << p1.y << endl;

    return 0;
}
```

- The values inside the braces are assigned to data members **in order**:
  - first value → first member (`x`)
  - second value → second member (`y`)
- Aggregate initialization also works with simple classes that have **public data members and no user-declared constructors**:

```
#include <iostream>
using namespace std;

class Color
{
public:
    int r;
    int g;
    int b;
};

int main()
{
    Color c1 = {255, 128, 64};

    cout << c1.r << " " << c1.g << " " << c1.b << endl;

    return 0;
}
```

- However, if a class has a **user-defined constructor**, then aggregate initialization is no longer used in the same way (it will prefer constructors instead).
- Example difference:

```
struct A
{
    int x;
    int y;
};

A a1 = {1, 2};   // OK: aggregate initialization

class B
{
public:
    B(int a, int b) {}
};

B b1 = {1, 2};   // calls constructor, NOT aggregate initialization
```

- Aggregate initialization is useful because it provides a concise way to initialize simple data structures without writing constructors.
- It is commonly used in:
  - simple structs
  - POD (plain old data) types
  - configuration-like data containers

9.12 constexpr with Classes and Objects

- We learned `constexpr` in Lesson 3 where it offers a powerful way to **improve performance** by computing the expression at **compile-time**.
- By marking your **functions** that operate on **constants** or **const-expression** as `constexpr`, we are instructing the compile to evalute those functions and **insert their result** instead of inserting instructions that compute the result when the applicatin is executed.
- This keyword can also be used with **classes** and **objects** that evaluate as **constants** as demonstrated by **Listing 9.18** constexpr with class Human.
- Note the **compiler** would **ignore** the **constexpr** when the function or class is used with entities that are **not constant**.

9.13 QA

- **Q: What is the difference between the instance of a class and an object of that class?**

  A: Essentially none. When you **instantiate** a class, you get an **instance** that can also be called an **object**.

- **Q: What is a better way to access members: using the dot operator (.) or using the pointer operator ($->$)?**

  A: If you have a **pointer** to an **object**, the **pointer operator** would be best suited. If you have instantiated an **object** as a **local variable** on the stack, then the **dot operator** is best suited.

- **Q: Why is the parameter of a copy constructor one that takes the copy source by reference?**

  A: For one, the **copy constructor** is **expected** by the compiler that way. The reason behind it is that a **copy constructor** would invoke itself it it accepted the copy source **by value**, resulting in an endless **copy loop**.

9.14 Quiz

1. My class has a raw pointer `int*` that contains a dynamically allocated array of integers. Does `sizeof` report different sizes depending on the number of integers in the dynamic array?

   A:

   - No. The `sizeof` operator reports only the size of the pointer itself, not the size of the dynamically allocated memory that it points to.
   - The dynamically allocated array resides on the **heap**, while the pointer is simply a variable stored as part of the class object.
   - For example:

     ```
     class Numbers
     {
     private:
         int* data;
     };
     ```

   - On a typical 64-bit system:

     ```
     sizeof(Numbers) == 8
     ```

     because the class contains only a single pointer, which is typically 8 bytes in size.

   - Even if the pointer refers to arrays of different lengths:

     ```
     Numbers a;   // data points to 5 integers
     Numbers b;   // data points to 100 integers
     Numbers c;   // data points to 1,000,000 integers
     ```

     the result of `sizeof` is the same for every object:

     ```
     sizeof(a) == sizeof(b) == sizeof(c)
     ```

   - This is because `sizeof` measures only the memory occupied by the class object itself, not any memory that has been dynamically allocated on the heap.

10 Implementing Inheritance

- ***Object-oriented programming*** is based on 4 important aspects:
  1. encapsulation
  2. abstraction
  3. inheritance
  4. polymorphism

10.1 Basics of Inheritance

- ***Inheritance*** is an object-oriented programming (OOP) feature that allows one class to **acquire** the data members and member functions of another class.
- The class being **inherited from** is called the ***base class*** (also called the **parent** or **superclass**).
- The class that **inherits from** another class is called the ***derived class*** (also called the **child** or **subclass**).
- Inheritance promotes **code reuse** by allowing a derived class to reuse functionality already implemented in its base class.
- A derived class may:
  - Inherit existing data members.
  - Inherit existing member functions.
  - Add new data members.
  - Add new member functions.
  - Override inherited virtual functions.
- The general syntax for inheritance is:

```
class DerivedClass : access-specifier BaseClass
{
    // additional members
};
```

- The access specifier is commonly one of:
  - `public`
  - `protected`
  - `private`
- Public inheritance is the most common form and models an **"is-a"** relationship.
- Example:

```
#include <iostream>
#include <string>
using namespace std;

class Animal
{
public:
    void Eat()
```

```
    {
        cout << "Animal is eating." << endl;
    }
};

class Dog : public Animal
{
public:
    void Bark()
    {
        cout << "Dog is barking." << endl;
    }
};

int main()
{
    Dog pet;

    pet.Eat();   // inherited from Animal
    pet.Bark();  // defined in Dog

    return 0;
}
```

- Output:

```
Animal is eating.
Dog is barking.
```

- In the example above:
  - `Animal` is the base class.
  - `Dog` is the derived class.
  - The `Dog` object inherits the `Eat()` member function.
  - The `Dog` class also defines its own member function, `Bark()`.
- Conceptually, inheritance forms a hierarchy:

```
Animal
   |
   +---- Dog
```

- Since every `Dog` object is also an `Animal` object, a `Dog` can use all accessible members inherited from `Animal` in addition to its own members.
- Inheritance is one of the fundamental mechanisms that enables polymorphism and code reuse in object-oriented programming.

10.1.1 Access specific keyword protected

- Here is a mechanism that allows **derived class members** to modify chosen attributes of the **base class**, while **denying access** to the same from everyone else.
- `protected` keyword to the rescue.

> - **protected**, like **public** and **private**, is also an ***access specifier***.
> - When you delcare a **class attribute** or **function** as **protected**, you are effectively making it accessible to classes that **derive** (and **friends**), yet simulateneously making it **inaccessible** to everyone else **outside the class**, including **main()**

- `protected` is the **access specifier** you shoulder use if you want a certain attribute in a **base class** to be **accessible** to classes that **derive** from this **base** class, as demonstrated in **Listing 10.2**.

10.1.2 Base Class Initialiation — Passing Parameters to the Base Class

- What if a **base class** were to contain an **overloaded constructor** that requires arguments at the time of **instantiation**?
- How would such a **base class** be **instantiated** when the **derived class** is being **constructed**?
- The clue lies in using **initialization lists** and in invoking the appropriate **base class constructor** via the **constructor** of the **derived class** as shown in the following code:

```
class Base
{
public:
 Base(int someNumber) // overloaded constructor
 {
  // use someNumber
 }
};

class Derived: public Base
{
public:
 Derived() : Base(25) // instantiate Base with argument 25
 {
  // derived class constructor code
 }
};
```

- This is nice because the base class can esnure **every derived class** is forced to engage with the specific constructor as shown in **Listing 10.3**
- In this `Fish` now has a **constructor** that takes a **default parameter** initialzing `Fish::isFreshWaterFish`.

  Thus, the **only possibility** to create an **object** of `Fish` is via providing it a **parameter** that initialized the **protected member**.
- This way `class Fish` ensures that the **protected member** doesn't contain a **random value**, especially if a **derive class** forgets to set it.
- The **derived classes** are now **forced** to define a **constructor** that instantiates the **base class instance** of `Fish` with the right parameter.

10.1.3 Derived Class Overriding Base Class's Methods

- if a `class Derived` implements the same functions with the same return values and signatures as in a `class Base` it inherits from, it effectively **overrides** that method in `class Base` as shown:

```
class Base
{
public:
 void DoSomething() {}
};

class Derived : public Base
{
public:
 void DoSomething() {}
};
```

- Thus, if **method** `DoSomething()` were to be invoked using an **instance** of `Derived`, the it would **not** invoke the functionality in `class Base`.
- **Listing 10.4** Derived classes Overriding Base method

10.1.4 Invoking Overriden Methods of a Base Class

- In **Listing 10.4** we saw a **derived class overriding** a function from the **base class** by implementing its versio of the same signature.
- If you instead want to invoke the base's method, you have to use ***scope resolution operator (::)***

```
myDinner.Fish::Swim();  // invokes Fish::Sim() using instance of tuna
       // Fish being the base class and Tuna being the child class
```

- **Listing 10.5** Invoking Base class functions from derived class using ::

10.1.5 Derived Class Hiding Base Class's Methods

- **Overriding** can take an extreme form where `Tuna::Swim()` can potentially **hide** all **overloaded** verions of `Fish::Swim()` available, even causing compilation failure when the **overloaded** ones are used (hence, called ***hidden***), as demonstrated by **Listing 10.6**.
- We hide the function in the base class from the compiler.
- So, if you want to invoke a **base function** like `Fish::Swim(bool)` via an **instance of Tuna (child class)**, you have the following solutions

  1. Solution 1: Use the **scope resolution operator** in `main()`

     ```
     myDinner.Fish::Swim();
     ```

  2. Solution 2: Use keying **using** in `class Tuna` to **unhide** `Swim()` in `class Fish`:

     ```
     class Tuna : public Fish
     {
     public:
      using Fish::Swim;  // unhide all Swim() methods in class Fish

      void Swim()
      {
       cout << "Tuna swims real fast" << endl;
      }
     };
     ```

  3. Solution 3: **Override** all **overloaded variants** of `Swim()` in `class Tuna` (invoke methods of `Fish::Swim(...)` via `Tuna::Fish(...)` if you want):

     ```
     class Tuna : public Fish
     {
     public:
      void Swim(bool isFreshWaterFish)
      {
       Fish::Swim(isFreshWaterFish);
      }

      void Swim()
      {
       cout << "Tuna swims real fast" << endl;
      }
     };
     ```

10.2 Order of Construction

- When an object of a derived class is created, C++ constructs the **base class portion first**, followed by the **derived class portion**.
- This ensures that the base class is fully initialized before the derived class begins constructing its own data members.
- The order of construction is:
  1. Base class constructor.
  2. Derived class constructor.
- Example:

```
#include <iostream>
using namespace std;

class Animal
{
public:
    Animal()
    {
        cout << "Animal constructor" << endl;
    }
};

class Dog : public Animal
{
public:
    Dog()
    {
        cout << "Dog constructor" << endl;
    }
};

int main()
{
    Dog pet;

    return 0;
}
```

- Output:

```
Animal constructor
Dog constructor
```

- Although the object being created is a `Dog`, the `Animal` portion must be constructed first because every `Dog` object contains an `Animal` subobject.
- A useful way to visualize the process is:

```
Create Dog object
        |
        v
Construct Animal
        |
        v
Construct Dog
```

10.3 Order of Destruction

- Object destruction occurs in the **reverse order** of construction.
- When a derived object is destroyed, the derived class destructor executes first, followed by the base class destructor.
- The order of destruction is:
  1. Derived class destructor.
  2. Base class destructor.
- Example:

```
#include <iostream>
using namespace std;

class Animal
{
public:
    Animal()
    {
        cout << "Animal constructor" << endl;
    }

    ~Animal()
    {
        cout << "Animal destructor" << endl;
    }
};

class Dog : public Animal
{
public:
    Dog()
    {
        cout << "Dog constructor" << endl;
    }

    ~Dog()
    {
        cout << "Dog destructor" << endl;
    }
};

int main()
{
    Dog pet;

    return 0;
}
```

- Output:

```
Animal constructor
Dog constructor
Dog destructor
Animal destructor
```

- The derived class is destroyed first because it may depend on resources owned by the base class. The base class is destroyed only after the derived class has finished cleaning up.
- A useful way to visualize destruction is:

```
Destroy Dog object
        |
        v
Destroy Dog
        |
        v
Destroy Animal
```

- In summary:
  - Construction proceeds from the base class to the derived class.
  - Destruction proceeds from the derived class back to the base class.

10.4 Private Inheritance

- ***Private inheritance*** differs from ***public inheritance*** in that the inherited members become **private** members of the derived class (unless they were already private in the base class).
- The general syntax is:

```
class Derived : private Base
{
    // ...
};
```

- With private inheritance:
  - Public members of the base class become private in the derived class.
  - Protected members of the base class also become private in the derived class.
  - Private members of the base class remain inaccessible to the derived class.
- Example:

```
#include <iostream>
using namespace std;

class Animal
{
public:
    void Eat()
    {
        cout << "Animal is eating." << endl;
    }
};

class Dog : private Animal
{
public:
    void MakeDogEat()
    {
        Eat();     // OK: inherited as private
    }
};

int main()
{
    Dog pet;

    pet.MakeDogEat();

    // Compile error:
    // pet.Eat();

    return 0;
}
```

- Output:

```
Animal is eating.
```

- Although `Eat()` was originally a public member of `Animal`, it becomes a private member of `Dog` because of private inheritance.

- Therefore:
  - The `Dog` class itself can call `Eat()`.
  - Code outside of `Dog` cannot call `Eat()` through a `Dog` object.
- Unlike public inheritance, private inheritance does **not** model an **"is-a"** relationship.
- Instead, private inheritance is sometimes used to express an implementation relationship, where the derived class reuses the base class's implementation without exposing its interface publicly.
- In modern C++, composition is generally preferred over private inheritance unless there is a specific reason to inherit privately.

10.5 Composition (side note)

- ***Composition*** is an object-oriented programming technique in which one class contains an object of another class as one of its data members.
- Composition models a **"has-a"** relationship.
- Inheritance models a **"is-a"** relationship.
- For example, a `Car` **has an** `Engine`, so composition is an appropriate design choice.

```
class Engine
{
public:
    void Start()
    {
        cout << "Engine started." << endl;
    }
};

class Car
{
private:
    Engine engine;    // Car has an Engine

public:
    void StartCar()
    {
        engine.Start();
    }
};
```

- By contrast, inheritance models an **"is-a"** relationship.
- For example, a `Dog` **is an** `Animal`, so inheritance is appropriate.

```
class Animal
{
public:
    void Eat() {}
};

class Dog : public Animal
{
};
```

- As a rule of thumb:
  - Use **inheritance** when there is an **"is-a"** relationship.
  - Use **composition** when there is a **"has-a"** relationship.
- This is why modern C++ encourages the design principle:

  "Favor composition over inheritance."

- In many situations, composition leads to code that is easier to understand, more flexible, and simpler to maintain.

10.6 Protected Inheritance

- ***Protected inheritance*** is another form of inheritance in which the inherited `public` and `protected` members of the base class become `protected` members of the derived class.
- The general syntax is:

```
class Derived : protected Base
{
    // ...
};
```

- With protected inheritance:
  - Public members of the base class become protected.
  - Protected members of the base class remain protected.
  - Private members of the base class are still inaccessible.
- Example:

```
#include <iostream>
using namespace std;

class Animal
{
public:
    void Eat()
    {
        cout << "Animal is eating." << endl;
    }
};

class Dog : protected Animal
{
public:
    void MakeDogEat()
    {
        Eat();    // OK
    }
};

class Puppy : public Dog
{
public:
    void Feed()
    {
        Eat();    // OK because Eat() is protected in Dog
    }
};

int main()
{
    Dog pet;
    pet.MakeDogEat();

    // Compile error:
    // pet.Eat();

    Puppy puppy;
```

```
    puppy.Feed();

    return 0;
}
```

- Output:

```
Animal is eating.
Animal is eating.
```

- In this example:
  - The `Dog` class can call `Eat()`.
  - The `Puppy` class can also call `Eat()` because it inherits from `Dog`.
  - Code outside these classes cannot call `Eat()` through a `Dog` or `Puppy` object.
- Therefore, protected inheritance allows derived classes to continue using inherited functionality while preventing direct access from outside the class hierarchy.
- The access transformations are summarized below:

```
Base member      After protected inheritance
-------------------------------------------
public           protected
protected        protected
private          inaccessible
```

- Protected inheritance is used less frequently than public inheritance. It is most useful when a class wishes to reuse a base class's implementation while allowing only further derived classes to access the inherited members.

- | Private and protected signifiy a "has-a" relationship |

10.7 Multiple Inheritance

- ***Multiple inheritance*** allows a class to inherit from **more than one base class**.
- This enables a derived class to reuse the functionality of multiple independent classes.
- The general syntax is:

```
class Derived : public Base1, public Base2
{
    // ...
};
```

- Example:

```
#include <iostream>
using namespace std;

class Printer
{
public:
    void Print()
    {
        cout << "Printing..." << endl;
    }
};

class Scanner
{
public:
    void Scan()
    {
        cout << "Scanning..." << endl;
    }
};

class AllInOne : public Printer, public Scanner
{
};

int main()
{
    AllInOne device;

    device.Print();
    device.Scan();

    return 0;
}
```

- Output:

```
Printing...
Scanning...
```

- In this example:
  - `Printer` is a base class.
  - `Scanner` is another base class.

  - `AllInOne` inherits from both classes.

- As a result, an `AllInOne` object has access to the member functions of both base classes.
- One issue that can arise with multiple inheritance is ***name ambiguity***. For example:

```
class A
{
public:
    void Hello()
    {
        cout << "Hello from A" << endl;
    }
};

class B
{
public:
    void Hello()
    {
        cout << "Hello from B" << endl;
    }
};

class C : public A, public B
{
};

int main()
{
    C obj;

    // obj.Hello();   // Compile error: ambiguous

    obj.A::Hello();
    obj.B::Hello();

    return 0;
}
```

- Since both base classes define a member function named `Hello()`, the compiler cannot determine which one should be called.
- The ambiguity can be resolved using the ***scope resolution operator*** (`::`), as shown above.
- Multiple inheritance is a powerful language feature, but it should be used carefully because it can make class hierarchies more complex.
- One well-known issue associated with multiple inheritance is the **Diamond Problem**, which is discussed separately using ***virtual inheritance***.

10.8 Avoid Inheritance via `final`

- The `final` keyword prevents a class from being used as a **base class**.
- Any attempt to derive from a class marked `final` results in a compile-time error.
- The general syntax is:

```
class ClassName final
{
    // ...
};
```

- Example:

```
#include <iostream>
using namespace std;

class Animal final
{
public:
    void Speak()
    {
        cout << "Animal speaks." << endl;
    }
};

// Compile error:
// class Dog : public Animal
// {
// };

int main()
{
    Animal pet;
    pet.Speak();

    return 0;
}
```

- Output:

```
Animal speaks.
```

- In the example above, `Animal` is declared as `final`, so no other class may inherit from it.
- Attempting to write:

```
class Dog : public Animal
{
};
```

  results in a compile-time error because `Animal` is a `final` class.

- The `final` keyword is commonly used to:
  - Prevent further inheritance.
  - Protect a class from unintended extension.
  - Indicate that the class represents a complete implementation.

- The `final` keyword can also be applied to **virtual member functions**.
- When a virtual function is declared `final`, derived classes may inherit the function, but they cannot override it.

```
class Base
{
public:
    virtual void Print() final
    {
        cout << "Base::Print()" << endl;
    }
};

class Derived : public Base
{
    // Compile error:
    // void Print() override { }
};
```

- In summary:
  - Applying `final` to a class prevents inheritance.
  - Applying `final` to a virtual function prevents overriding.

11 Polymorphism

11.1 Basics of Polymorphism

- **"Poly"** is Greek for ***many*** and **"morph"** means ***form***.
- ***Polymorphism*** is the feature of **object-oriented** programming that allows **objects** of different types to be accessed through a common interface, with each object providing its own implementation of the same operation.
- We go over **polymorphic** behavior that can be implemented in C++ via the **inheirtance hierarchy** known as ***subtype polymorphism***.

11.1.1 Need for Polymorphic Behavior

- Last section we sa how `Tuna` and `Carp` **inheirt public methods** `Swim()` from `Fish` (Listing 10.1)
- It is, however, possible that both `Tuna` and `Carp` provide their **own** `Tuna::Swim()` and `Carp::Swim()` methods to make `Tuna` and `Carp` different swimmers.
- If a user with an **instance** of `Tuna` uses the **base class type** to invoke `Fish::Swim()`, he ends up executing only the **generic part** `Fish::Swim()` and **not** `Tuna::Swim()`, even though that **base class instance** `Fish` is a part of a `Tuna`.
- **Listing 11.1** Invoking Methods Using an Instance of the Base Class Fish That Belongs to a Tuna
- Code shows if we cast it back to a `Fish`, `Tuna` behaves like a generic Fish and not a Tuna.
- What the user would **ideally expect** is that an **object** of **type** `Tuna` behaves like a tuna **even if** the methods invoked is `Fish::Swim()`.
- We use ***virtual functions*** to fix this.

11.1.2 Polymorphic Behavior Implemented Using Virtual Functions

- You have access to an **object** of **type** `Fish`, via **pointer** `Fish*` or **reference** `Fish&`.

  This **object** could have been **instantiated** solely as `Fish` or be part of a `Tuna` or `Carp` that **inheirts** from `Fish`.

- You don't know (or don't give a fuck)
- You invoke **method** `Swim()` using the **pointer** or **reference** like this:

```
pFish->Swim();
myFish.Swim();
```

- What you expect is that the **object** `Fish` swims
  - as a `Tuna` if it is part of a `Tuna`,
  - as a `Carp` if it is part of a `Carp`,
  - or an anonymous `Fish` if it wasn't instantiatead as part of a specialized class such as `Tuna` or `Carp`.
- You can ensure this by **declaring function** `Swim()` in the **base class** as a ***virtual function***

```
class Base
{
 virtual ReturnType FunctionName (Parameter List);
};

class Derived
{
 ReturnType FunctionName (Parameter List);
};
```

- Use of the keyboard `virtual` means that the **compiler** ensures that any **overriding variant** of the requested **base class method** is invoked.
- Thus, if `Swim()` is declared **virtual**, invoking `myFish.Swim()` (myFish being of **type** `Fish&`) results in `Tuna::Swim()` being executed (Listing 11.2)
- This is ***polymorphism***: treating different fishes as a common type `Fish`, yet **ensuring** that the right implementation of `Swim()` supplied by the **derived types** is executed

11.1.3 Need For Virtual Destructors

- What happens when a function calls operator `delete` using a **pointer** of type `Base*` that actually points to the **instance of type Derived**?
- See **Listing 11.3** A function that invokes operator 'delete' on Base*
- Note in the listing: while `Tuna` and `Fish` were both **constructed** on the **free store (heap)** due to `new`, the **destructor** of `Tuna` **was not invoked** during `delete`, a rather only that of the `Fish`.
- This is in **stark contrast** to the **construction** and **destruction** of **local member** `myDinner` where **all** the **constructors** and **destructors** are invoked.
- Clearly, for the **free store** we have amiss. `~Tuna()` needs to be invoked.

```
Allocating a Tuna on the free store:
Constructed Fish
Constructed Tuna
Deleteing the Tuna:
Destructed Fish
Instantiating a Tuna on the stack:
Constructed Fish
Constructed Tuna
Automatic destruction as it goes out of scope:
Destructed Tuna
Destructed Fish
```

- The flaw means that the **destructor** of a **deriving class** that has been instantiated on the **free store** using `new` wwould **not** be invoked if `delete` is called using a **pointer of type** `Base*`.
- This can result in resources not being released, memory leaks, and so on and is a problem that is not to be taken lightly.
- **Listing 11.4** Using virtual destructors to ensure that destructors in derived classes are invoked when deleting a pointer of Type Base*

```
Allocating a Tuna on the free store:
Constructed Fish
Constructed Tuna
Deleteing the Tuna:
Destructed Tuna
Destructed Fish
Instantiating a Tuna on the stack:
Constructed Fish
Constructed Tuna
Automatic destruction as it goes out of scope:
Destructed Tuna
Destructed Fish
```

- **Always declare the base class destructor as 'virtual'**

```
class Base
{
public:
 virtual ~Base() {};  // virtual destructor
};
```

- This ensures that one with a **pointer** `Base*` cannot invoke `delete` in a way that instances of the **deriving classes** are not correctly destroyed.

11.1.4 How do Virtual Functions Work? — The Virtual Function Table (vtable)

- A ***virtual function*** allows **runtime polymorphism**. When a member function is declared with the `virtual` keyword, the function that is executed is determined by the **actual type of the object** rather than the type of the pointer or reference.
- To support this behavior, most C++ compilers implement a hidden data structure called the ***Virtual Function Table (vtable)***. Although the C++ standard does **not** require this implementation, it is the method used by virtually all modern compilers.
- The **vtable** is a table of function pointers. Each class that contains one or more virtual functions has its own vtable.
- Every object of a class containing virtual functions secretly stores a hidden pointer called the ***vptr*** (virtual pointer). This pointer points to the class's vtable.
- During object construction, the compiler automatically initializes the object's `vptr` so that it points to the correct vtable for that object's type.
- When a virtual function is called through a pointer or reference:
  1. The program follows the object's hidden `vptr`.
  2. The `vptr` points to the appropriate vtable.
  3. The corresponding function pointer is retrieved from the table.
  4. The function pointed to is invoked.
- This process is known as ***dynamic dispatch*** or ***late binding***, since the decision of which function to execute is made at **runtime** instead of compile time.
- Without virtual functions, C++ performs ***static binding*** (also called ***early binding***), meaning the compiler decides which function to call solely from the pointer or reference type.
- Consider the following example:

```
class Fish
{
public:
    virtual void Swim()
    {
        cout << "Fish swims\n";
    }
};

class Tuna : public Fish
{
public:
    void Swim() override
    {
        cout << "Tuna swims fast\n";
    }
};

Fish* pFish = new Tuna;
pFish->Swim();
```

- Even though `pFish` is a `Fish*`, the object it points to is actually a `Tuna`. Since `Swim()` is virtual, the compiler uses the object's `vptr` to locate Tuna's implementation in the vtable, producing:

```
Tuna swims fast
```

- If `virtual` were omitted, the compiler would instead call:

  ```
  Fish swims
  ```

  because it would only consider the pointer's declared type (`Fish*`).

- Conceptually, the compiler generates structures similar to the following (greatly simplified):

```
Fish vtable
+----------------------+
| &Fish::Swim          |
| &Fish::~Fish         |
+----------------------+

Tuna vtable
+----------------------+
| &Tuna::Swim          |
| &Tuna::~Tuna         |
+----------------------+
```

- Each object contains a hidden pointer:

```
    Fish object

+------------------+
| vptr ----------+ |
+----------------|--+
| data members   |  |
+----------------+  |
                    |
                    v
           Fish vtable
         +------------------+
         | Fish::Swim       |
         | Fish::~Fish      |
         +------------------+
```

- If the object is actually a `Tuna`, its hidden `vptr` instead points to Tuna's vtable:

```
    Tuna object

+------------------+
| vptr ----------+ |
+----------------|--+
| Fish members   | |
| Tuna members   | |
+----------------+ |
                    |
                    v
           Tuna vtable
         +------------------+
         | Tuna::Swim       |
         | Tuna::~Tuna      |
         +------------------+
```

- Since the `vptr` belongs to the object itself, the correct virtual functions are called regardless of whether the object is accessed through a base-class pointer, base-class reference, or directly.

- Virtual destructors also use the vtable. When deleting an object through a base-class pointer:

  ```
  Fish* pFish = new Tuna;
  delete pFish;
  ```

  the virtual destructor ensures the call sequence is:

  ```
  Tuna::~Tuna()
  Fish::~Fish()
  ```

  If the destructor were not virtual, only `Fish:: Fish()` would be called, resulting in **undefined behavior** and potentially leaking resources owned by the derived class.

- Although virtual functions introduce a very small runtime overhead (one extra pointer lookup and one hidden pointer per object), this cost is generally negligible compared to the flexibility provided by runtime polymorphism.

- **Summary:**
  - Each polymorphic class has one vtable.
  - Each object stores one hidden `vptr`.
  - Virtual function calls use the `vptr` to locate the correct function in the vtable.
  - Function selection occurs at runtime (dynamic dispatch).
  - Virtual destructors ensure complete destruction of derived objects when deleting through base-class pointers.

11.2 Abstract Base Classes and Pure Virtual Functions

- A **base class** that **cannot be instantiated** is called an ***abstract base class***.
- Such **base class** fulfills only **one purpose**, that of **being derived from**.
- C++ allows you to create an ***abstract base class*** using ***pure virtual functions***

- A ***virtual method*** is said to be ***pure virtual*** when it has a **declaration** as shown in the following:

```
class AbstractBase
{
public:
 virtual void DoSometing() = 0;  // pure virtual method
};
```

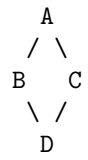
- This **declaration** essentially tells the **compiler** that `DoSomething()` **needs to be implemented** and by **class that derives** from `AbstractBase`:

```
class Derived : public AbstractBase
{
public:
 void DoSomething()
 {
  cout << "Implemented virtual function" << endl;
 }
};
```

- Thus, what the class `AbstractBase` has done is that it has **enforced** class `Derived` to **supply an implementation** for **virtual method** `DoSomething()`.
- This functionality where a **base class** can **enforce** support of methods with a **specified name and signature** in **classes** that **derive** from it is that of an ***interface***.
- So, making `class Fish` an ***abstract base class*** with `Swim()` as a ***pure virtual function*** **ensures** that `Tuna` that **derives** from `Fish` implements `Tuna::Swim()` and swims like a Tuna and not like just any Fish.
- **Listing 11.6** class Fish as an Abstract Base Class for Tuna and Carp

11.3 Using 'virtual' Inheritance to Solve the Diamond Problem

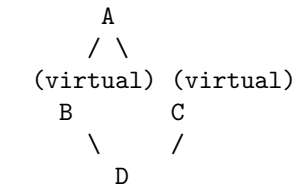
- The ***diamond problem*** occurs in **multiple inheritance** when a class inherits from two classes that both inherit from the same base class.
- This creates a diamond-shaped inheritance structure:



- In this structure:
  - Class `A` is the base class.
  - Classes `B` and `C` both inherit from `A`.
  - Class `D` inherits from both `B` and `C`.
- The problem arises because `D` ends up containing **two copies of `A`**:
  - One inherited through `B`
  - One inherited through `C`
- This leads to ambiguity. For example, if `A` has a member variable or function, calling it from `D` becomes unclear:

```
D obj;
obj.A::function();  // ambiguous: which A?
```

- It also wastes memory since two separate copies of `A`'s data exist inside `D`.
- C++ solves this problem using ***virtual inheritance***.
- When `B` and `C` virtually inherit from `A`, they share a single common instance of `A` inside `D`.



- This is written in code as:

```
class A
{
public:
    void foo() {}
};

class B : virtual public A {};
class C : virtual public A {};

class D : public B, public C {};
```

- Now, `D` contains only **one shared instance of A**, eliminating duplication and ambiguity.
- Without virtual inheritance, the compiler cannot decide which `A` subobject to use when accessing members through `D`, resulting in a compilation error.

- With ***virtual inheritance***, the **most derived class** (`D`) is **responsible** for constructing the **shared base class** `A`.
- Example:

```
class A
{
public:
    int value;
};

class B : virtual public A {};
class C : virtual public A {};

class D : public B, public C {};

int main()
{
    D obj;
    obj.value = 10;  // OK: only one A exists
}
```

- **Key idea:**
  - Virtual inheritance ensures that only one copy of a shared base class exists in the final derived object.
  - It resolves ambiguity and prevents duplicate state.
- **Tradeoff:**
  - Virtual inheritance introduces a small runtime and memory overhead because the shared base class is accessed indirectly.
  - However, it is necessary when designing complex inheritance hierarchies.

- **Listing 11.8** Demonstrating How 'virtual' keyword in Inheritance Heirarchy helps restrict the number of instances of base class Animal to One

- In C++, `public virtual Animal` and `virtual public Animal` are **equivalent**. The order of the specifiers does not affect the meaning.
- Both forms indicate:
  - **public inheritance** (controls access level)
  - **virtual inheritance** (ensures a single shared base class instance in diamond inheritance)
- The C++ grammar allows **multiple valid orders of inheritance specifiers** for flexibility, so both are accepted by the compiler.
- There is no difference in behavior or memory layout between the two forms.
- **Style convention:** many modern C++ guidelines prefer `public virtual Base` because it reads as "publicly inherits from Base" with virtual inheritance as an additional detail.

```
class B : public virtual A {};   // ok
class B : virtual public A {};   // ok
class B : protected virtual A {}; // ok
class B : virtual protected A {}; // ok
class B : private virtual A {};   // ok
class B : virtual private A {};   // ok
```

11.4 Specifier 'override' to Indicate Intention to Override

- The `override` specifier in C++ is used to explicitly indicate that a member function is intended to **override a virtual function** from a base class.
- It improves code safety by letting the compiler verify that:
  - The base class actually has a matching virtual function.
  - The function signature in the derived class correctly matches the base class version.
- If there is no matching virtual function in the base class, the compiler will produce an error.
- This helps prevent subtle bugs caused by accidental mismatches in function signatures.
- Example without `override` (bug-prone):

```
class Animal
{
public:
    virtual void speak()
    {
        cout << "Animal sound\n";
    }
};

class Dog : public Animal
{
public:
    void speak(int x)   // does NOT override Animal::speak()
    {
        cout << "Bark\n";
    }
};
```

- In this case, `Dog::speak(int)` is a completely new function, not an override, because the signature does not match.
- Using `override` catches this mistake:

```
class Dog : public Animal
{
public:
    void speak(int x) override   // compiler error
    {
        cout << "Bark\n";
    }
};
```

- The compiler will reject this because there is no matching virtual function in the base class with the same signature.
- Correct usage:

```
class Dog : public Animal
{
public:
    void speak() override
    {
        cout << "Bark\n";
    }
};
```

- **Benefits of `override`:**
  - Prevents accidental signature mismatches.
  - Improves code readability by making intent explicit.
  - Helps the compiler catch polymorphism errors early.
- **Key idea:** `override` does not change runtime behavior — it is purely a **compile-time safety check**.

11.5 Using 'final' to Prevent Function Overriding

- The `final` specifier in C++ is used to **prevent further overriding** of a **virtual function** or **prevent inheritance** from a **class**.
- When applied to a virtual function, `final` ensures that no derived class can override that function.
- Example:

```
class Animal
{
public:
    virtual void speak() final
    {
        cout << "Animal sound\n";
    }
};
```

- In this case, any attempt to override `speak()` in a derived class will result in a compiler error:

```
class Dog : public Animal
{
public:
    void speak() override   // error: cannot override final function
    {
        cout << "Bark\n";
    }
};
```

- **Key idea:** marking a function as `final` "locks" its implementation in the inheritance hierarchy.
- `final` can also be applied to an entire class to prevent inheritance altogether:

```
class Animal final
{
public:
    void speak()
    {
        cout << "Animal sound\n";
    }
};
```

- In this case, any attempt to derive from the class is illegal:

```
class Dog : public Animal   // error: cannot inherit from final class
{
};
```

- **Benefits of `final`:**
  - Prevents unintended modification of critical behavior.
  - Allows the compiler to optimize virtual dispatch in some cases.
  - Clearly communicates design intent to other developers.
- **Key idea:** `final` is a compile-time restriction that strengthens class and interface design by limiting extensibility.

11.6 Virtual Copy Constructors?

- The question mark here is because it is technically impossible in C++ to have **virtual copy constructors**.
- **Virtual copy constructors** are **not possible** because the concept of `virtual` functions in C++ relies on **runtime polymorphism**, where a function call is resolved based on the **dynamic type of an already-constructed object**.
- A constructor, however, is fundamentally different: it is responsible for **creating an object from scratch**. At the time a constructor runs, there is **no fully constructed object yet**, and therefore no vtable-based dispatch can occur.
- In other words, `virtual` functions require an existing object with a **vptr** (virtual table pointer), but during construction the object is still being built, so polymorphic dispatch cannot apply.
- Additionally, constructors are **not inherited in the same way as normal member functions** and are always called based on the **static type being constructed**, not the dynamic type.
- **Copy constructors specifically** must create a new object of a known type, so their signature and target type are fixed at compile time. There is no meaningful way to "choose" a derived constructor at runtime.
- For this reason, C++ does not allow constructors (including copy constructors) to be declared as `virtual`.
- **Key idea:** virtual dispatch requires an existing object and runtime type information, while construction is a compile-time process that produces the object in the first place. These two mechanisms are fundamentally incompatible.

- Having said that, there is a nice workaround in the form of defining your own clone function that allows you to do just that:

```
class Fish
{
public:
 virtual Fish* Clone() const = 0; // pure virtual function
};

class Tuna : public Fish
{
// ... other members
public:
 Tuna * Clone() const // virtual clone function
 {
  return new Tuna(*this); // return new Tuna that is a copy of this
 }
};
```

- Thus, **virtual function** `Clone` is a ***smulated virtual copy constructor*** that needs to be explicitly invoked, as shown in **Listing 11.9**

11.7 QA

11.8 Quiz

12 Operator Types and Operator Overloading

- In addition to encapsulating data and methods, classes can also encapsulate **operators** that make it easy to operate on **instances** of this class.
- You can use these **operators** to perform operations such as **assignment** or **addition** on **class objects** similar to those on integers that you saw in Lesson 5.
- Just like **functions**, **operators** can also be overloaded.

12.1 What are Operators in C++?

- On a syntactical level, there is very little that differentiates an **operator** from a **function**, save for the use of the keyword `operator`.
- Here is an ***operator declaration***:

```
return_type operator operator_symbol (...parameter list...);
```

- The `operator_symbol` in this case could be any of the several **operator types** that the programmer can define.
- It could be
  - + (addition)
  - && (logical AND)
  - and so on...
- The ***operands*** help the compiler distinguish **one operator** from **another**.
- So, why does C++ provide operators when functions are supported?

- Consider a utility class `Date` that encapsulates the day, month, and year:

```
Date holiday(12, 25, 2016); // Dec 26, 2016
```

- Assuming that you want to **add a day** and get the **instance** to contain the next day — Dec 26, 2016 — which of the following two options would be more intuitive?
  1. Option 1 (using the **increment operator** ++):

     ```
     ++holiday;
     ```

  2. Option 2 (using a **member function** `Increment()`):

     ```
     holiday.Increment(); // Dec 26, 2016
     ```

- Clearly, **Option 1** scores over method `Increment()`
- The **operator-based mechanism** facilitates consumption by supplying ease of use and intuitiveness.

- Implementing **operator** $<$ in class `Date` would help you compre **two instances** of `class Date` like this:

```
if (date1 < date2)
 do something;
else
 do something else;
```

- **Operators** can be used in more situations than just **classes** that manage dates.
- An **addition operator** $(+)$ in a **string utility class** such as `MyString` introduced in Listing 9.9 would facilitate easy **concatentation**:

```
MyString sayHello("Hello ");
MyString sayWorld(" world");
MyString sumThem (sayHello + sayWorld);   // if operator+ were supported by MyString
```

- The effor in implementing relevant operators will be rewarded by the ease of consumption of the class.
- On a broad level, **operators** in C++ can be classified into two types:
  1. ***Unary operators***
  2. ***Binary Operators***

12.2 Unary Operators